



RV College of Engineering®

Mysore Road, RV Vidyaniketan Post, Bengaluru - 560059, Karnataka, India

PyVulSev: A Benchmark for Adversarial Robustness in Python Vulnerability Detection

MAJOR PROJECT REPORT (CS481P)

Submitted by

Saara Unnathi R

1RV22CS167

Under the guidance of

Dr. Manonmani S
Associate Professor
Dept. of CSE
RV College of Engineering

**Dr. Holger Schmidt &
Dr. Klaus Kaiser**
Professor
Computer Science
Fachhochschule Dortmund

In partial fulfillment for the award of degree of
Bachelor of Engineering
in
Computer Science and Engineering

**Department of CSE
2025-26**





RV College of Engineering®

Mysore Road, RV Vidyaniketan Post, Bengaluru - 560059, Karnataka, India

Go, change the world

PyVulSev: A Benchmark for Adversarial Robustness in Python Vulnerability Detection

A Major Project Report (CS481P)

Submitted by

Saara Unnathi R

1RV22CS167

Under the guidance of

Dr. Manonmani S

Associate Professor

Dept. of CSE

RV College of Engineering

Dr. Holger Schmidt &

Dr. Klaus Kaiser

Professor

Computer Science

Fachhochschule Dortmund

In partial fulfillment of the requirements for the degree of

Bachelor of Engineering in

Computer Science and Engineering

2025-26

RV College of Engineering®, Bengaluru
(Autonomous institution affiliated to VTU, Belagavi)
Department of Computer Science and Engineering



CERTIFICATE

Certified that the major project (CS481P) work titled ***PyVulSev: A Benchmark for Adversarial Robustness in Python Vulnerability Detection*** is carried out by **Saara Unnathi R (1RV22CS167)** who is bonafide student of RV College of Engineering, Bengaluru, in partial fulfillment of the requirements for the degree of **Bachelor of Engineering in Computer Science and Engineering** of the Visvesvaraya Technological University, Belagavi during the year 2025-26. It is certified that all corrections/suggestions indicated for the Internal Assessment have been incorporated in the major project report deposited in the departmental library. The major project report has been approved as it satisfies the academic requirements in respect of major project work prescribed by the institution for the said degree.

Project Guide

Dr. Manonmani S

**Head of the
Department**

Dr. Shanta
Rangaswamy

Dean CS Cluster

Dr. Ramakanth
Kumar P

Principal

Dr. K. N. Subramanya

External Viva Examination

Name of Examiner

Signature with Date

- 1.
- 2.

DECLARATION

I, **Saara Unnathi R** student of eighth semester B.E., Department of Computer Science and Engineering, RV College of Engineering, Bengaluru-560059, bearing USN: **1RV22CS167**, hereby declare that the major project titled '**PyVulSev: A Benchmark for Adversarial Robustness in Python Vulnerability Detection**' has been carried out by me and submitted in partial fulfilment for the award of degree of **Bachelor of Engineering in Computer Science and Engineering** during the year 2025-26. Further I declare that the content of the dissertation has not been submitted previously by anybody for the award of any degree or diploma to any other university.

I also declare that any Intellectual Property Rights generated out of this project carried out at RVCE will be the property of RV College of Engineering, Bengaluru and I will be one of the authors of the same.

Place: Bengaluru

Date:

Name

Signature

1. Saara Unnathi R(1RV22CS167)

ACKNOWLEDGEMENT

I am indebted to my guide, **Dr. Manonmani S**, Associate Professor, Department of CSE, RVCE for the suggestions & advice through the project work.

I extend my sincere gratitude to **Prof. Dr. Holger Schmidt** and **Prof. Dr. Klaus Kaiser**, Department of Computer Science, Fachhochschule Dortmund, for their invaluable guidance, and for welcoming this collaboration from RV College of Engineering.

My sincere thanks to the project coordinators **Dr. Chethana R Murthy**, Associate Professor, Department of CSE and **Prof. Deepika Dash**, Assistant Professor, Department of CSE for their timely instructions and support in coordinating the project.

My gratitude to **Prof. Narashimaraja P**, Department of ECE, RVCE for the organized latex template which made report writing easy.

My sincere thanks to **Dr. Shanta Rangaswamy**, Professor and Head, Department of Computer Science Engineering, RVCE and **Dr. Ramakanth Kumar P**, Professor and Dean (CSE Cluster), RVCE for the support and encouragement.

I express sincere gratitude to our beloved **Dr. M.V. Renukadevi**, Professor and Dean Academics, RVCE, **Dr. Geetha K S**, Professor and Vice Principal, RVCE and **Dr. K. N. Subramanya**, Principal, RVCE for the appreciation towards this project work.

I thank all the teaching staff and technical staff of Computer Science Engineering department, RVCE for their help. Lastly, I take this opportunity to thank my family members and friends who provided all the backup support throughout the project work.

Saara Unnathi R

Computer Science and Engineering,
RV College of Engineering,
Bengaluru-59.

ABSTRACT

Automated Python vulnerability detectors are evaluated almost entirely on labels inherited from Common Vulnerabilities and Exposures (CVE) fix commits, where empirical audits report 25% to 50% label noise from co-changed files. A benchmark built on such labels measures a detector’s agreement with label noise as much as its detection ability. The CASTLE benchmark addressed this for C and C++ with a hand-curated corpus, but no comparable evaluation substrate existed for Python, which is the dominant language for security tooling, machine learning pipelines, and modern web applications. A second gap compounded the first: detection accuracy was reported on unmodified code only, so a reported score was an upper bound that might not survive a trivial, behavior-preserving rewrite of the input.

This project constructed a benchmark for Python vulnerability detection covering seven sink-shaped Common Weakness Enumeration (CWE) classes, with 869 samples made up of 440 audit-validated real-world positives and 429 synthesized safe hard negatives. Labels were validated by a three-layer methodology: a pre-ingest sink-presence filter, a three-round adversarial audit by independent reviewers, and a diff-based filter that requires the sink line itself to change between the vulnerable and fixed versions. The benchmark was paired with seven semantics-preserving adversarial mutators across three families, each targeting one detector shortcut, and seven detectors were evaluated across four tiers: pattern-matching static analysis, whole-project dataflow, frontier large language models (LLMs), and a fine-tuned GraphCodeBERT classifier.

The combined validation pipeline reduced the audited false-positive rate from 52% to 26%. Across the seven detectors, no single tool led on every measure, and the sink-token mutator family separated pattern-matching static analysis, where Bandit lost 0.187 macro-F1 under attribute-access obfuscation, from a learned detector that was invariant to every mutator in the suite. All artifacts, audit reports, rejection manifests, and tool runners were released to support replication.

LIST OF PUBLICATIONS

Publication Type	Publication Details
International Conference	Saara Unnathi R, “A Sink-Validated Python Vulnerability Detection Benchmark with Adversarial Robustness Probes”, manuscript prepared for submission to an IEEE international conference, 2026.
Journal	Saara Unnathi R, “A Sink-Validated Python Vulnerability Detection Benchmark with Adversarial Robustness Probes”, manuscript prepared for submission to <i>Computers & Security</i> (Elsevier), 2026.

The work in this report has been written up in two forms: a condensed conference version prepared against the IEEE conference template, and an extended journal version prepared against the Elsevier *Computers & Security* template. Both report the same audit-validated benchmark, the three-layer label-validation methodology, and the seven-detector, four-tier evaluation described in this report.

CONTENTS

Abstract	i
List of Publications	ii
List of Figures	vii
List of Tables	ix
Abbreviations	x
List of Symbols	xi
1 Introduction	1
1.1 Overview of Project Topic	2
1.1.1 Automated Vulnerability Detection	2
1.1.2 Global Scenario	3
1.1.3 Societal Relevance	3
1.1.4 Problem Area	3
1.2 Specific Details of the Project Topic	3
1.3 Literature Review	4
1.4 Problem Statement	4
1.5 Objectives	4
1.6 Scope	5
1.7 Methodology	5
1.8 Organization of the Report	5
2 Theory and Concept	6
2.1 Sink-Shaped Vulnerability Classes	7
2.2 Detector Families	7
2.3 The Label-Noise Problem	8
2.4 Semantics-Preserving Robustness	8
2.5 Evaluation Measures	9
2.6 Related Work	9

2.6.1	Datasets	9
2.6.2	Label Noise and Validation	9
2.6.3	Detectors and Robustness	9
2.6.4	Positioning	10
3	Software Requirement Specification	11
3.1	Overall Description	12
3.1.1	Product Perspective	12
3.1.2	Product Functions	12
3.1.3	Constraints	13
3.2	Specific Requirements	13
3.2.1	Functional Requirements	13
3.2.2	Performance Requirements	14
3.2.3	Software Requirements	14
3.2.4	Hardware Requirements	15
3.2.5	Design Constraints	15
3.3	Summary	15
4	High Level Design	16
4.1	Design Considerations	17
4.1.1	General Considerations	17
4.1.2	Development Method	17
4.2	Architecture Strategies	17
4.2.1	Programming Language	17
4.2.2	Future Plans	18
4.3	System Architecture	18
4.4	Data Flow Diagrams	18
4.4.1	Data Flow Diagram Level 0.0	18
4.4.2	Data Flow Diagram Level 1.0	19
4.4.3	Data Flow Diagram Level 2.0	19
4.5	Summary	19
5	Detailed Design	21
5.1	Structure Chart	22

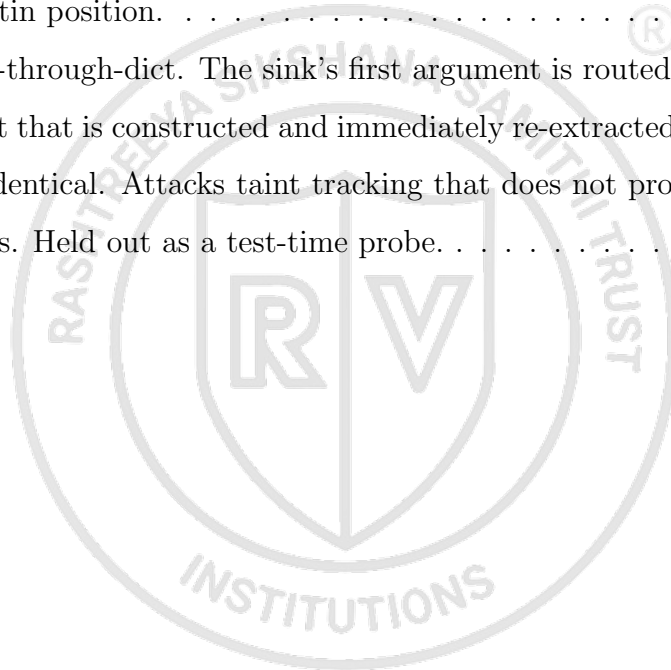
5.2	Module Description	22
5.2.1	Validation Modules	22
5.2.2	Negative Builder	23
5.2.3	Mutator Suite	23
5.2.4	Metrics Module	24
5.2.5	Detector Runners	24
5.3	Summary	24
6	Implementation	26
6.1	Programming Language Selection	27
6.2	Platform Selection	27
6.3	Code Conventions	27
6.3.1	Naming Convention	27
6.3.2	Artifact Schema	28
6.4	Difficulties Encountered and Strategies Used to Tackle Them	28
6.5	Data Set	29
6.5.1	Layer 1: Sink-Presence Filter	29
6.5.2	Layer 2: Adversarial Audit	30
6.5.3	Layer 3: Diff-Based Filter	30
6.5.4	Final Composition	31
6.5.5	Hard-Negative Construction	32
6.6	Algorithms Used	32
6.6.1	Mutator Transformations	32
6.6.2	Trained Detector	33
6.7	Summary	33
7	Software Testing	34
7.1	Testing Strategy	35
7.2	Component Testing	35
7.2.1	Sink-Presence Filter	35
7.2.2	Diff Filter	36
7.2.3	Negative Builder	36
7.2.4	Mutator Semantics Preservation	36

7.3	Pipeline Testing	37
7.4	Validation Testing through the Audit	37
7.5	Test Cases	38
7.6	Summary	38
8	Experimental Results	39
8.1	Research Questions and Setup	40
8.2	Results Analysis	40
8.2.1	Headline Results	40
8.2.2	Detection on Clean Code (RQ1)	41
8.2.3	Adversarial Robustness (RQ2)	41
8.2.4	Per-CWE Validation Progression	42
8.3	Detailed Analysis	43
8.3.1	LLM Detectors	43
8.3.2	Trained-Model Detector	44
8.3.3	Whole-Project Dataflow Detectors	44
8.3.4	Effect of the Label Filter (RQ3)	44
8.4	Summary	44
9	Conclusion	45
9.1	Limitations	46
9.2	Future Enhancement	47
A	Plagiarism report	48
B	Publication details	50
C	Code: Adversarial Mutators	52
C.1	Sink-Preserving Family (M1 to M4)	53
C.2	Sink-Targeted Family (M5, M6)	54
C.3	Dataflow Family (M7)	55
	Bibliography	56

LIST OF FIGURES

4.1	The three-layer label-validation pipeline. Layer 1 rejects samples lacking the category-defining sink token; Layer 2's independent audit rounds drive the pattern tightenings; Layer 3 keeps a positive only if the sink line itself changed between the vulnerable and fixed versions.	18
4.2	Data Flow Diagram Level 0.0 (context diagram).	19
4.3	Data Flow Diagram Level 1.0: corpus construction, mutation, and evaluation.	19
4.4	Data Flow Diagram Level 2.0: the 129-sample benchmark and eight mutator variants of the 67 positives flow into seven detectors across four tiers, scored under five bounded metrics.	20
5.1	The seven mutators grouped by the detector shortcut each family probes. Mutators marked † are held out of training-time augmentation and used only as test-time robustness probes.	24
6.1	Layer-1 sink-presence filter (simplified). A sample is admitted only if a CWE sink pattern matches after comment and docstring stripping, subject to the per-CWE proximity gates.	30
6.2	Layer-3 diff filter. A positive passes if at least one sink line in <code>code_before</code> is absent from <code>code_after</code> . Whitespace is normalized so that reformat-only edits do not count as real changes.	31
8.1	Per-CWE audit false-positive rate across the three validation rounds (x-axis labels are CWE numbers; <i>All</i> is the aggregate). The diff filter collapses the co-changed-file classes to 0% while the residual concentrates on CWE-89 and CWE-79.	43
C.1	M1 dead-code injection. Two side-effect-free statements are added; the return value cannot change. Attacks positional reasoning.	53
C.2	M2 string-literal splitting. The substring "SELECT" no longer appears in the source, but the runtime SQL string is byte-identical. Attacks substring matching. Held out as a test-time probe.	53

C.3	M3 identifier rename. Locals and parameters are renamed to synonyms in all their references. Attacks identifier memorization.	54
C.4	M4 wrapper extraction. The sink call is moved into a one-line nested helper that forwards arguments. Attacks single-function analysis.	54
C.5	M5 sink attribute obfuscation. An attribute sink <code>value.attr(args)</code> becomes <code>getattr(value, "attr")(args)</code> . Attacks token-level sink matching for CWE-78, 89, 918, and 502.	54
C.6	M6 sink via globals. A builtin sink <code>eval(arg)</code> is rewritten through a <code>builtins</code> dictionary lookup, as shown above, so the sink token survives only as a string literal. Attacks the same token-level match on the CWE-94 bare-builtin position.	55
C.7	M7 taint-through-dict. The sink's first argument is routed through a one-entry dict that is constructed and immediately re-extracted, so the value is bitwise identical. Attacks taint tracking that does not propagate through dict reads. Held out as a test-time probe.	55



LIST OF TABLES

6.1	Positive data sources and counts after label validation. Hard negatives are constructed safe twins.	29
6.2	Final per-CWE composition of the benchmark.	32
7.1	Representative test cases for the benchmark pipeline.	38
8.1	Detector results on the 129-sample clean test set. Macro-F1 is one-versus-rest across seven CWEs; MCC is on the binary vulnerable/safe collapse; SWR is severity-weighted recall on the 27 CVSS-scored positives; HCMA is hierarchical accuracy; κ_w is weighted Cohen's κ ; Comp. is macro-F1 on the composed mutator; $\Delta F1^*$ is the worst-case single-mutator drop (positives only).	41
8.2	Per-CWE F1 on the clean test set (n = positive test samples). GCB is the trained GraphCodeBERT detector.	42
8.3	Macro-F1 by variant over the 67 positives. DC/SS/VR/WE: sink-preserving; SAO/SVG: sink-targeted; TTD: dataflow; Comp: composed.	42
8.4	Robustness drop (clean – mutator) by family. Sink-preserve is the mean of the four sink-preserving mutators; SAO, SVG, and TTD are individual. ΔSWR is the drop in severity-weighted recall.	43

ABBREVIATIONS

API Application Programming Interface

AST Abstract Syntax Tree

CVE Common Vulnerabilities and Exposures

CVSS Common Vulnerability Scoring System

CWE Common Weakness Enumeration

GHSA GitHub Security Advisory

GPU Graphics Processing Unit

HCMA Hierarchical CWE Macro-Accuracy

HTML HyperText Markup Language

HTTP HyperText Transfer Protocol

LDAM Label-Distribution-Aware Margin

LLM Large Language Model

MCC Matthews Correlation Coefficient

NVD National Vulnerability Database

OSV Open Source Vulnerabilities database

OWASP Open Worldwide Application Security Project

RAM Random Access Memory

RQ Research Question

SAST Static Application Security Testing

SQL Structured Query Language

SSRF Server-Side Request Forgery

SWR Severity-Weighted Recall

XSS Cross-Site Scripting

LIST OF SYMBOLS

$\Delta\mathbf{F1}$ Robustness drop: clean macro-F1 minus mutated macro-F1

$\Delta\mathbf{SWR}$ Drop in severity-weighted recall under a mutator

† Mutator held out of training-time augmentation and used only as a test-time probe

κ_w Weighted Cohen’s kappa, a chance-corrected agreement score

n Number of positive test samples in a CWE class





Chapter 1

Introduction

CHAPTER 1

INTRODUCTION

Software security depends on finding vulnerabilities in source code before an attacker does. Automated vulnerability detection tools aim to do this at scale, flagging the lines of code where an injection, a traversal, or an unsafe deserialization can occur. The accuracy of such a tool is only as trustworthy as the benchmark used to measure it, and most current benchmarks inherit their labels from records that were never written to mark code as vulnerable. This chapter introduces the project domain of automated Python vulnerability detection, sets out its global and societal relevance, identifies the gaps that motivate the work, and states the objectives, scope, methodology, and organization of the report.

1.1 Overview of Project Topic

The project builds and evaluates a benchmark for Python vulnerability detection. A benchmark in this setting is a labeled corpus of code samples paired with a scoring protocol, against which detectors are run so that their results can be compared on common ground. The work covers two axes that prior benchmarks treat in isolation: the correctness of the labels, and the robustness of a detector when the input code is rewritten without changing what it does.

1.1.1 Automated Vulnerability Detection

A vulnerability detector reads source code and predicts whether it contains a security flaw, and which class of flaw. The classes follow the Common Weakness Enumeration (CWE) catalog, for example SQL injection (CWE-89) or cross-site scripting (CWE-79). Detectors fall into distinct families. Pattern matching static application security testing (SAST) tools such as Bandit and Semgrep scan a single file for known unsafe constructs. Whole-project dataflow tools such as CodeQL and Snyk Code trace data from an input source to a dangerous sink across an entire application. Large language models (LLMs) reason over code in natural language, and learned classifiers such as GraphCodeBERT are fine-tuned on labeled vulnerability data. Each family makes a different trade-off between coverage, precision, and the context it requires.

1.1.2 Global Scenario

Most research in this area is built on C and C++. The standard datasets, BigVul, CVEfixes, CrossVul, DiverseVul, Devign, and ReVeal, are dominated by C and C++ code and contribute few Python samples per CWE class. Python is the dominant language for security tooling, machine learning pipelines, and modern web applications, yet it has no curated, audit-validated detection benchmark comparable to the CASTLE benchmark that exists for C and C++. The result is a mismatch between where security-sensitive Python code is written and where detection methods are measured.

1.1.3 Societal Relevance

Web applications and data pipelines written in Python process personal data, financial records, and critical infrastructure traffic. A single missed injection or deserialization flaw can expose an entire user base. Detection tools shift the cost of finding these flaws from the attacker to the defender, but only if their reported accuracy reflects real detection ability rather than agreement with mislabeled data. A benchmark with validated labels and a robustness axis gives practitioners a more honest basis for choosing a tool, which raises the security floor of the software that society depends on.

1.1.4 Problem Area

Two problems define the gap this project addresses. First, the labels in existing benchmarks are noisy. They are inherited from CVE fix commits, and a fix commit touches several files for refactoring, tests, or cleanup that have nothing to do with the vulnerability. Each co-changed file inherits the CWE label of the advisory, and manual audits report between 25% and 50% label noise as a result [1], [2]. A benchmark built on raw inheritance therefore measures a detector’s agreement with label noise as much as its detection ability [3]. Second, accuracy is reported on unmodified code only. A reported score is an upper bound that may not survive a trivial, behavior-preserving rewrite of the input [4], so it says little about how a detector behaves on code it has not seen in that exact form.

1.2 Specific Details of the Project Topic

The benchmark covers seven *sink-shaped* CWE classes drawn from the MITRE Top-25 [5]: SQL injection (CWE-89), cross-site scripting (CWE-79), path traversal (CWE-22), OS command injection (CWE-78), code injection (CWE-94), server-side request

forgery (CWE-918), and insecure deserialization (CWE-502). A sink-shaped class has a well-defined unsafe function, the sink, whose misuse is the vulnerability and whose fix modifies the sink call. Structural classes that encode the absence of a check, such as missing authorization, require application-level context and are excluded on methodological grounds.

The corpus holds 869 samples: 440 real-world positives validated by audit and 429 synthesized safe hard negatives. Labels pass through a three-layer validation methodology, and the corpus is paired with seven semantics-preserving mutators that probe detector shortcuts. Seven detectors across four tiers are evaluated on both clean and mutated code.

1.3 Literature Review

The literature divides into four threads that this project unifies. Large vulnerability datasets establish the supply of labeled code but are C and C++ centric [6], [7]. Studies of label noise, by Croft et al., the ReVeal authors, CleanVul, and PrimeVul, quantify how inherited CVE labels mislead detector evaluation [1], [2], [3], [8]. Comparative studies such as CASTLE pit static analyzers against LLMs but rely on CVE-inherited labels and add no robustness axis [9]. Robustness studies, including SecLLMHolmes, show that semantics-preserving edits can flip a detector’s verdict [4]. Chapter 2 reviews these threads in detail.

1.4 Problem Statement

To construct an audit-validated benchmark for Python vulnerability detection across sink-shaped CWE classes, with a label-validation methodology that reduces inherited label noise, and to measure detector accuracy and adversarial robustness jointly so that the evaluation distinguishes detectors that reason about code from detectors that pattern-match its surface.

1.5 Objectives

- Assemble a Python vulnerability corpus across seven sink-shaped CWE classes from advisory and CVE-fix sources, paired with constructed safe hard negatives.
- Design and apply a three-layer label-validation methodology that lowers the audited false-positive rate of the labels.

- Build a suite of semantics-preserving mutators, each targeting one detector shortcut, to probe robustness.
- Evaluate seven detectors across four tiers on clean and mutated code under a set of bounded, complementary metrics.
- Release all artifacts, audit reports, and rejection manifests to support replication.

1.6 Scope

The benchmark is Python-specific and limited to sink-shaped CWE classes. It does not cover structural CWEs, multi-language code, or whole-application analysis, and the scores are not a comprehensive ranking of Python security tools. The validation methodology is language-agnostic in design, but its sink pattern set would need re-deriving for another language. The trained detector is presented as a small-data existence proof rather than a state-of-the-art claim.

1.7 Methodology

The work proceeds in four stages. First, positives are scraped from five advisory and CVE-fix sources and passed through a sink-presence filter, an independent adversarial audit, and a diff-based filter that anchors each positive on a modified sink line. Second, safe hard negatives are constructed as remediated twins of real vulnerable functions. Third, seven mutators are implemented over the Python abstract syntax tree (AST), each deterministic and semantics-preserving. Fourth, seven detectors are run on the held-out test set, clean and mutated, and scored under five bounded metrics. Each stage is reproducible from the released artifacts.

1.8 Organization of the Report

Chapter 2 presents the theory and concepts behind vulnerability detection, label noise, and robustness, together with related work. Chapter 3 specifies the functional and non-functional requirements of the benchmark system. Chapter 4 gives the high-level design, including the system architecture and data flow. Chapter 5 details the design of each module. Chapter 6 describes the implementation, including the dataset pipeline, mutators, and trained detector. Chapter 7 covers the testing of the pipeline and harness. Chapter 8 reports the experimental results across the seven detectors. Chapter 9 concludes with limitations and future enhancements.



Chapter 2

Theory and Concept

CHAPTER 2

THEORY AND CONCEPT

This chapter sets out the concepts the benchmark is built on. It defines the vulnerability classes in scope, the families of detectors that the benchmark scores, the label-noise problem that motivates the validation methodology, the notion of semantics-preserving robustness, and the evaluation measures used to report results. It closes with a review of related work across datasets, label noise, detectors, and robustness studies.

2.1 Sink-Shaped Vulnerability Classes

A vulnerability class in the Common Weakness Enumeration (CWE) catalog describes a recurring kind of security flaw. This work targets seven *sink-shaped* classes, where a sink is a function whose unsafe use is the vulnerability and whose safe use is not. SQL injection (CWE-89) occurs when untrusted input reaches a database query sink without parameterization. Cross-site scripting (CWE-79) reaches an HTML rendering sink, OS command injection (CWE-78) reaches a process-execution sink, code injection (CWE-94) reaches `eval` or `exec`, path traversal (CWE-22) reaches a filesystem sink, server-side request forgery (CWE-918) reaches an outbound HTTP sink, and insecure deserialization (CWE-502) reaches a sink such as `yaml.load` or `pickle.loads`.

The sink-shaped property matters because it gives each positive an anchor: the fix for the vulnerability modifies the sink call. Structural classes, such as missing authorization or cross-site request forgery, encode the *absence* of a check and require application-level context to judge. They have no single line that a diff can anchor on, so they are excluded from this benchmark on methodological grounds rather than as a statement about their importance.

2.2 Detector Families

The benchmark scores detectors from four tiers, each resting on a different principle.

- **Pattern-matching SAST.** Bandit [10] and Semgrep [11] scan a single file for syntactic signatures of unsafe constructs, in the same lexical family as general Python linters such as Pylint [12]. They are fast and need no build, but they reason about the surface form of code rather than its data flow.
- **Whole-project dataflow.** CodeQL [13] and Snyk Code track data from a source

to a sink across an entire application. They are more precise on complete programs but need a built database with application entry points.

- **Large language models.** LLMs reason over code in natural language and are competitive on C and C++ benchmarks [14], [15], but their verdicts can shift under small input changes, and code they generate carries security bugs of its own [16].
- **Learned classifiers.** A model such as GraphCodeBERT [17], [18] is fine-tuned on labeled vulnerability data and learns features from both token sequence and data-flow structure, following a line of deep vulnerability detectors that begins with token-level [19] and representation-learning [20] models.

2.3 The Label-Noise Problem

Detector training and evaluation depend on labels that say which code is vulnerable. Most labels are inherited from CVE fix commits. A fix commit touches several files for refactoring, tests, or cleanup that do not relate to the vulnerability, and each co-changed file inherits the CWE label of the advisory. Croft et al. [1] audited BigVul and reported about 25% noise; the ReVeal authors [8] showed that models learn co-changed surface patterns rather than the vulnerability itself. CleanVul [2] found 40% to 75% of labels inaccurate across common datasets, and PrimeVul [3] showed that a model scoring 68.3% F1 on BigVul drops to 3.1% on a deduplicated, tightened-label version of the same task. The lesson is that a benchmark built on raw label inheritance measures a detector’s agreement with label noise as much as its detection ability. The validation methodology in Chapter 6 addresses this with a sink-anchored, line-level criterion.

2.4 Semantics-Preserving Robustness

A reported detection score is measured on unmodified code. The robustness question asks whether that score survives a rewrite of the input that changes its surface form but not its runtime behavior. A semantics-preserving transformation is one that leaves the program’s observable behavior unchanged, for example renaming a local variable or splitting a string literal. If a detector’s verdict changes under such a transformation, the detector relied on a surface shortcut rather than on the program’s meaning. Se-cLLMHolmes [4] showed that such edits flip LLM verdicts, and the broader literature on code-model robustness [21], [22], [23] establishes that learned models are sensitive to

these edits. The mutators in this work differ by targeting one interpretable shortcut each, so a per-mutator accuracy drop is diagnostic rather than a single opaque attack score.

2.5 Evaluation Measures

Detection quality is reported with five bounded, complementary measures rather than a single composite, so a reader can see where a detector is strong. Macro averaged F1 [24] summarizes per-class precision and recall across the seven classes, which matters because class sizes are unequal. The Matthews correlation coefficient (MCC) [25] scores the binary vulnerable-versus-safe decision and holds up when the classes are balanced. Severity-Weighted Recall (SWR) weights recall by CVSS severity so that missing a critical flaw costs more than missing a minor one. Hierarchical CWE Macro Accuracy (HCMA) gives Wu-Palmer [26] partial credit for close-sibling confusions on a CWE subtree. Weighted Cohen’s κ [27] reports chance-corrected agreement. Robustness is reported as the drop in macro-F1 between clean and mutated code, computed over the same positives so the subtraction is like-for-like.

2.6 Related Work

2.6.1 Datasets

Source-code vulnerability benchmarks are C and C++ dominant. BigVul [6], CVE-fixes [28], CrossVul [29], and the deduplicated DiverseVul [7] contribute few per-class Python samples. Juliet [30] is synthetic and does not transfer to natural code [8]; Devign and ReVeal [8], [31] are C only. Within Python, `vudenc` [32] is the closest labeled corpus but ships no paired fixed versions, which constrains the label validation it admits.

2.6.2 Label Noise and Validation

Beyond the audits noted above, D2A [33] uses differential analysis for labels but operates at program rather than sink granularity. The diff-based filter in this work is a sink-anchored, line-level label-validity criterion, and to our knowledge it is the first such evidence-based criterion in this literature.

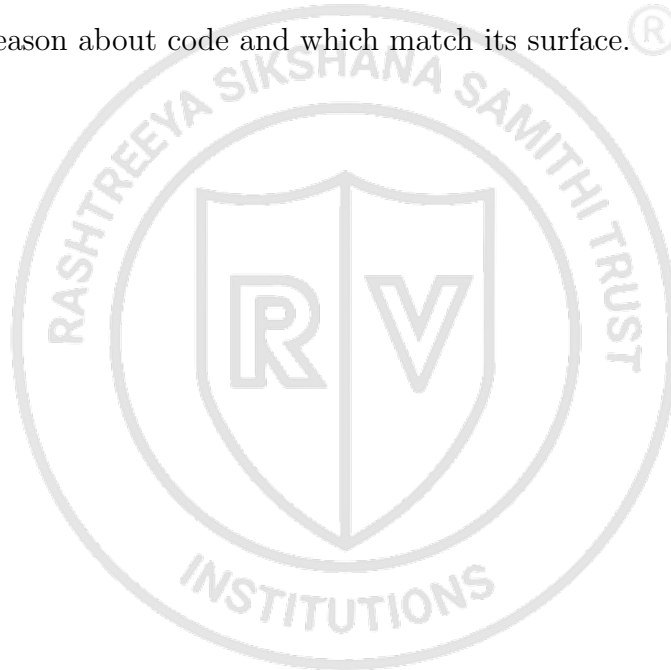
2.6.3 Detectors and Robustness

The most relevant prior comparison is CASTLE [9], which pits static analyzers against LLMs for C and C++ but uses CVE-inherited labels and adds no robustness axis. RealVuln [34] targets Python repositories with manual labels but reports detection only.

IRIS [35] is an LLM and CodeQL hybrid whose CWE-Bench-Java targets the same sink-shaped family. LineVul [36] and MoreFixes [37] extend the dataset and modeling line. Se-cLLMHolmes [4] established that semantics-preserving edits flip detector verdicts, which motivates the robustness axis here.

2.6.4 Positioning

Each ingredient has precedent. Large vulnerability datasets, the identification of CVE label noise, the SAST-versus-LLM comparison, and code-model robustness studies all exist on their own. This work joins them for Python: seven sink-shaped Top-25 CWE classes, an automated sink-anchored label-validity criterion, and a per-mutator robustness breakdown reported next to detection accuracy in one evaluation. That last step surfaces which detectors reason about code and which match its surface.®



Chapter 3

Software Requirement Specification

CHAPTER 3

SOFTWARE REQUIREMENT SPECIFICATION

This chapter specifies the requirements of the benchmark system: the corpus construction pipeline, the mutator suite, and the evaluation harness. It states the overall description of the system, its perspective and functions, and the functional, performance, software, and hardware requirements that the implementation satisfies. The chapter closes with a summary of the key specifications.

3.1 Overall Description

The system is a research pipeline, not an interactive application. It ingests raw vulnerability records, validates their labels, constructs safe counterparts, generates adversarial variants, runs a set of detectors, and emits scored results. Its users are security researchers and tool authors who need a trustworthy substrate to measure Python vulnerability detectors.

3.1.1 Product Perspective

The benchmark is a standalone, reproducible pipeline with three stages that hand off through versioned artifacts on disk. The first stage produces a validated corpus of labeled samples. The second stage produces mutated variants of the test samples. The third stage runs detectors and computes metrics. Each stage reads and writes schema-versioned files, so a stage can be re-run in isolation and any result can be traced back to the inputs that produced it. The system relates to external systems only as data sources: advisory databases and CVE feeds upstream, and detector binaries or APIs at evaluation time.

3.1.2 Product Functions

The system performs the following functions:

- Scrape positive samples from five advisory and CVE-fix sources.
- Filter samples by sink presence, removing those without the category-defining sink token.
- Support an independent, CWE-stratified audit of label correctness.

- Apply a diff-based filter that keeps a positive only if its sink line changed between the vulnerable and fixed versions.
- Construct safe hard negatives as remediated twins of real vulnerable functions.
- Split the corpus into train, validation, and test sets with no cross-split repository leakage.
- Generate seven semantics-preserving mutated variants of each test sample.
- Run seven detectors across four tiers and score them under five bounded metrics on clean and mutated code.

3.1.3 Constraints

The system operates under these constraints:

- Labels must be evidence-backed: a positive is admitted only with a matched sink pattern and, where a paired fix exists, a changed sink line.
- Every mutator must be semantics-preserving, and every mutated sample must re-parse as valid Python.
- Samples sharing a repository must fall in the same split, to prevent leakage.
- All validation decisions must be reproducible from released artifacts.

3.2 Specific Requirements

3.2.1 Functional Requirements

1. The scraper shall retrieve per-CWE positives with explicit fix commits where the source provides them.
2. The sink-presence filter shall admit a sample only if at least one regex sink pattern matches its source after comments and docstrings are stripped.
3. The audit interface shall present each item with its identifier, source, repository, file path, matched pattern, and code excerpt, and record a binary verdict with a one-sentence justification.

4. The diff filter shall compare the vulnerable and fixed versions after comment stripping and whitespace normalization, and reject a positive whose sink lines are unchanged.
5. The negative builder shall apply the canonical remediation for a CWE at the AST level, preserving surface structure and the sink name while neutralizing the sink behavior.
6. Each mutator shall transform the Python AST deterministically given a sample identifier and seed, and shall skip any node it cannot prove safe to transform.
7. The harness shall score every detector one-versus-rest per CWE and report macro-F1, MCC, SWR, HCMA, and weighted κ .

3.2.2 Performance Requirements

The pipeline runs offline in batch. The throughput requirement is that the full corpus, on the order of two thousand pre-filter samples, be processed end to end on a single workstation. Detector runs must complete within the rate limits of the hosted LLM APIs, which sets the practical bound on wall-clock time. The trained detector must fit and run on a single GPU. Determinism is required: the SAST tools and the fixed-checkpoint model produce identical output across runs, and the LLMs run at temperature zero to keep per-call drift small.

3.2.3 Software Requirements

- Python 3.11 or later for the pipeline, mutators, and harness.
- The Python standard `ast` module for AST transformations.
- PyTorch and the HuggingFace Transformers library for the trained detector.
- Bandit 1.9.4, Semgrep 1.163.0, and Snyk Code 1.1305.0 as SAST and dataflow detectors.
- API access to Claude Sonnet 4.6, GPT-4o, and DeepSeek R1 for the LLM tier.
- Git and SQLite for source retrieval (CVEfixes ships as a SQLite database).

3.2.4 Hardware Requirements

- A workstation with at least 16 GB of RAM for the pipeline and harness.
- A CUDA-capable GPU with at least 12 GB of memory to fine-tune the 126.6M-parameter GraphCodeBERT detector.
- Sufficient disk for the corpus, mutated variants, and per-detector prediction logs.

3.2.5 Design Constraints

The single-file extract format is a deliberate design constraint: each sample is a self-contained function, which suits SAST, LLM, and learned detectors but is unfavorable to whole-project dataflow tools that need application entry points. This trade-off is accepted so that the benchmark stays language-uniform and reproducible, and the whole-project tools are retained in the harness for a future application-level format.

3.3 Summary

The benchmark system is a three-stage, artifact-driven pipeline whose requirements center on evidence-backed labels, semantics-preserving mutation, leakage-free splits, and reproducible scoring. The functional requirements define each pipeline step, and the non-functional requirements fix the software and hardware needed to run it. These specifications guide the high-level design in Chapter 4.



Chapter 4

High Level Design

CHAPTER 4

HIGH LEVEL DESIGN

The high-level design describes the architecture of the benchmark system and the flow of data through it. This chapter states the design considerations and development method, the architecture strategy and language choice, the overall system architecture, and the data flow at successive levels of detail. It closes with a summary of the design framework.

4.1 Design Considerations

4.1.1 General Considerations

Four considerations shape the design. The first is reproducibility: every validation decision must be traceable to the inputs that produced it, so each stage reads and writes schema-versioned artifacts. The second is evidence: a positive is kept only with a matched sink pattern and a changed sink line. The third is isolation, which keeps the stages independent, so the corpus, the mutators, and the harness can be re-run on their own. The fourth is auditability. The pattern set, the audit reports, and the rejection manifest are released, which lets a reader check the label decisions instead of trusting them.

4.1.2 Development Method

The system was built incrementally and validated by audit at each round rather than once at the end. The three-round audit drove the design: each round measured the false-positive rate of the labels on disjoint samples, and the findings of one round motivated the filter added in the next. This audit-then-tighten loop is the development method, and it is why the false positive rate fell across rounds rather than being asserted at the start.

4.2 Architecture Strategies

4.2.1 Programming Language

Python is the implementation language for three reasons. The benchmark targets Python code, so the sink patterns, the AST transformations, and the semantics-preservation checks all operate in the same language they analyze. The standard `ast` module gives a stable, well-documented syntax tree, which the mutators and the negative builder transform directly. And the machine learning stack the trained detector needs, PyTorch and

Transformers, is native to Python.

4.2.2 Future Plans

The architecture leaves room for extension. The single-file format can be replaced by an application-level format that supplies entry points to the whole-project dataflow detectors, the sink pattern set can be re-derived for another language, and the mutator suite can gain a dynamic-dispatch axis. Each extension fits the existing stage boundaries without redesign.

4.3 System Architecture

The system is a pipeline of three stages connected by artifacts. The corpus stage narrows a raw feed through three validation layers; the mutation stage generates semantics-preserving variants of the test samples; and the evaluation stage runs detectors and computes metrics. Figure 4.1 shows the corpus stage, which is the part that establishes label quality. A raw feed of 1,982 samples is narrowed by the three layers, and the audited false-positive rate falls from 52% to 26%.

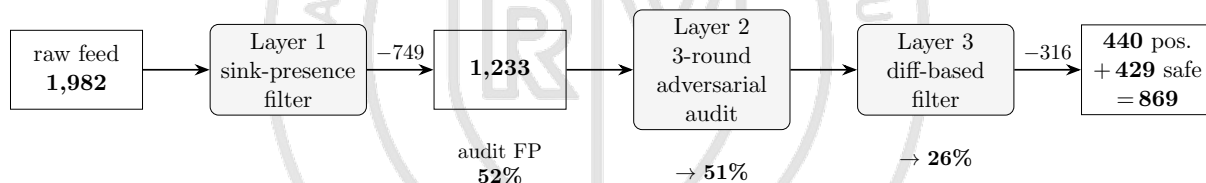


Figure 4.1: The three-layer label-validation pipeline. Layer 1 rejects samples lacking the category-defining sink token; Layer 2’s independent audit rounds drive the pattern tightenings; Layer 3 keeps a positive only if the sink line itself changed between the vulnerable and fixed versions.

4.4 Data Flow Diagrams

4.4.1 Data Flow Diagram Level 0.0

At the context level, the system takes two external inputs and produces one output. The inputs are the advisory and CVE-fix feeds (GHSA, CVEfixes, NVD, OSV, and the **vudenc** corpus) and the detector implementations (SAST binaries, LLM APIs, and the trained model). The output is a set of scored results: per detector, per CWE, on clean and mutated code. The system is the benchmark pipeline that transforms the feeds into a validated, mutated corpus and scores the detectors against it.

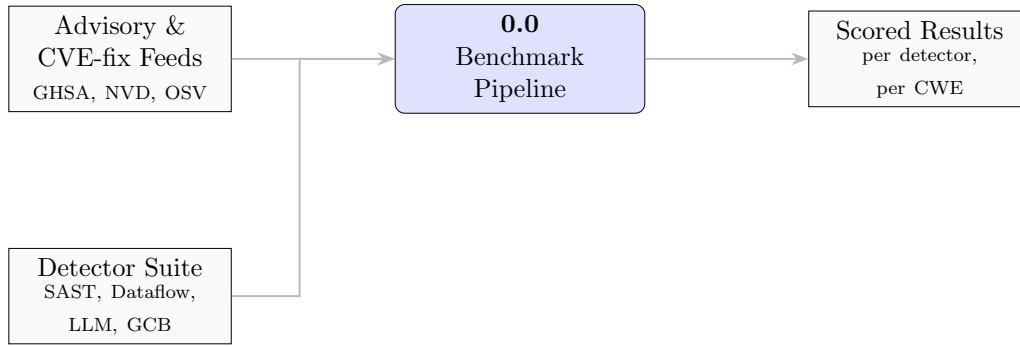


Figure 4.2: Data Flow Diagram Level 0.0 (context diagram).

4.4.2 Data Flow Diagram Level 1.0

One level down, the pipeline is three processes connected by data stores. The *construction* process reads the raw feeds, applies the three validation layers, builds the safe twins, and writes the validated corpus to a data store. The *mutation* process reads the test split and writes seven mutated variants of each test sample to a variant store. The *evaluation* process reads both the clean test set and the variant store, runs each detector, and writes per-sample predictions and aggregate metrics.

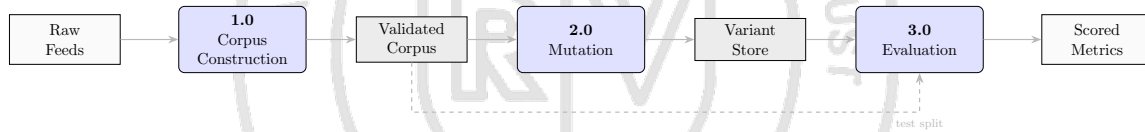


Figure 4.3: Data Flow Diagram Level 1.0: corpus construction, mutation, and evaluation.

4.4.3 Data Flow Diagram Level 2.0

The evaluation process expands into the harness shown in Figure 4.4. The same 129-sample clean set and eight mutator variants of the 67 positives flow into seven detectors across four tiers, and each detector’s output flows into the five bounded metrics.

4.5 Summary

The high-level design is a three-stage, artifact-connected pipeline built around reproducibility, evidence-backed labels, stage isolation, and auditability. The system architecture centers on the three-layer validation funnel, and the data flow refines from a two-input context diagram to the detailed evaluation harness. Chapter 5 details the design of each module.

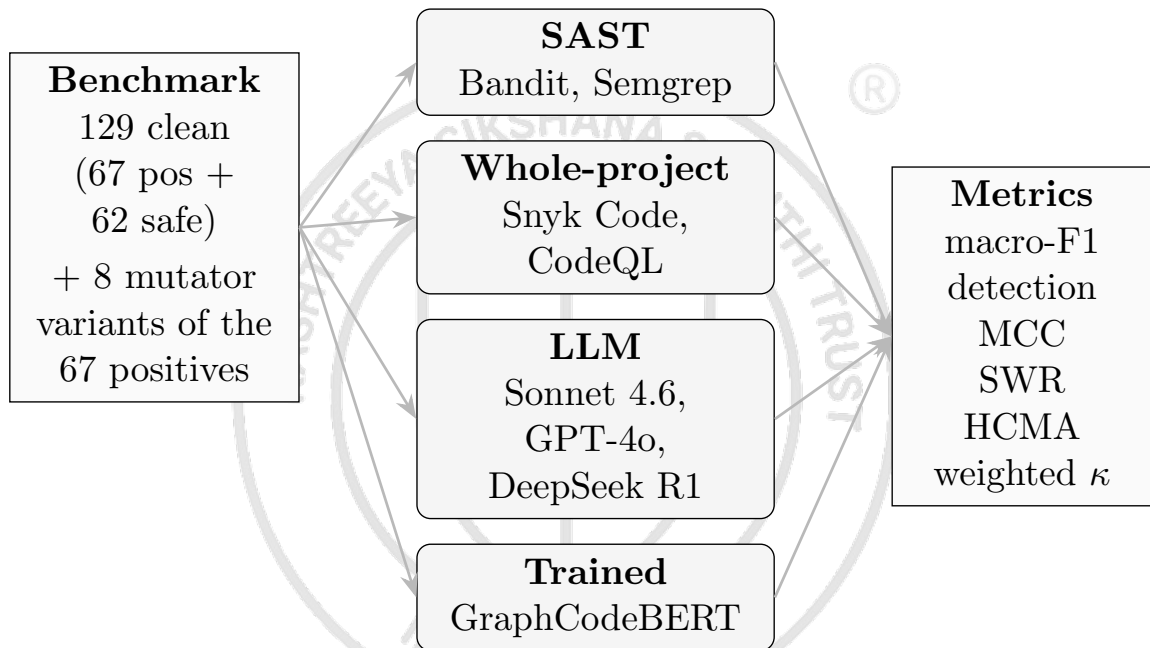


Figure 4.4: Data Flow Diagram Level 2.0: the 129-sample benchmark and eight mutator variants of the 67 positives flow into seven detectors across four tiers, scored under five bounded metrics.



Chapter 5

Detailed Design

CHAPTER 5

DETAILED DESIGN

The detailed design decomposes the benchmark system into modules and describes the internal behavior of each. This chapter presents the structure chart, then the design of the validation modules, the negative builder, the mutator suite, the metrics module, and the detector runners. It closes with a summary of the design specifications.

5.1 Structure Chart

The system divides into three subsystems, each a group of cooperating modules. The **corpus subsystem** holds the scraper, the sink-presence filter, the audit driver, the diff filter, the negative builder, and the splitter. The **mutation subsystem** holds the seven mutators and the semantics-preservation checker. The **evaluation subsystem** holds the detector runners, one per tool, and the metrics module. Control flows top down: the corpus subsystem produces the validated split, which the mutation subsystem consumes to produce variants, which the evaluation subsystem consumes alongside the clean split to produce scores. Data flows back up only as artifacts on disk, so no module holds state across runs.

5.2 Module Description

5.2.1 Validation Modules

The **sink-presence filter** defines, for each CWE, a small set of high-recall regex sink patterns. It admits a sample only if at least one pattern matches the source after comments and docstrings are stripped, which suppresses documentation matches. The patterns match both safe and unsafe forms of the sink, leaving discrimination to the diff filter. Two CWEs use proximity gates: CWE-22 requires an HTTP-request reference near the sink.

The **audit driver** presents CWE-stratified samples to an independent reviewer. For each item it supplies the identifier, source, repository, file path, the matched regex, and the code excerpt around the sink, and it records a binary correctly-labeled or mislabeled verdict with a one-sentence justification. Reviewers do not see each other's verdicts, so the audit is adversarial by construction.

The **diff filter** implements the evidence-anchored criterion. It compares the vulnera-

ble and fixed versions of a positive after comment stripping and whitespace normalization, and keeps the positive only if at least one line containing a sink-pattern match differs between the two versions. A positive whose sink line is unchanged is rejected, because the fix did not touch the sink and the label is therefore unsupported.

5.2.2 Negative Builder

The **negative builder** constructs the safe hard negatives. For each negative it takes a real vulnerable function and applies the canonical remediation for its CWE at the AST level, for example rewriting `yaml.load` to `yaml.safe_load` or a string-formatted SQL query to a parameterized query. The remediation preserves the surface structure and the sink *name* while neutralizing only the sink's behavior. This gives each negative a provable safe label and makes it a minimal-pair counterfactual: a detector that fires on the mere presence of a sink token is forced into a false positive on the twin.

5.2.3 Mutator Suite

The **mutator suite** generates seven semantics-preserving variants of each test sample. Every mutator follows three design principles. It is semantics-preserving: when a transformation cannot be proven safe, the mutator skips that node. It is targeted at one shortcut, so a per-mutator accuracy drop is diagnostic. And it is realistic: identifiers become synonyms and injected code reads like ordinary scaffolding. The mutators fall into three families, shown in Figure 5.1.

The *sink-preserving* family (M1 to M4: dead-code injection, string splitting, identifier rename, wrapper extraction) alters code around the sink but leaves the sink identifier and its arguments intact, probing positional, substring, identifier-memorization, and single-function shortcuts. The *sink-targeted* family rewrites the sink call itself: M5 turns `os.system(x)` into `getattr(os,"system")(x)` and M6 turns `eval(x)` into `__import__("builtins").__dict__["eval"](x)`, so the sink token survives only inside a string literal. The *dataflow* family (M7) routes the sink's first argument through a one-entry dict, `f({"a":x}["a"])`, breaking the static lineage from parameter to sink argument. At evaluation time two to three mutators are applied per sample, and each mutator is also reported in isolation.

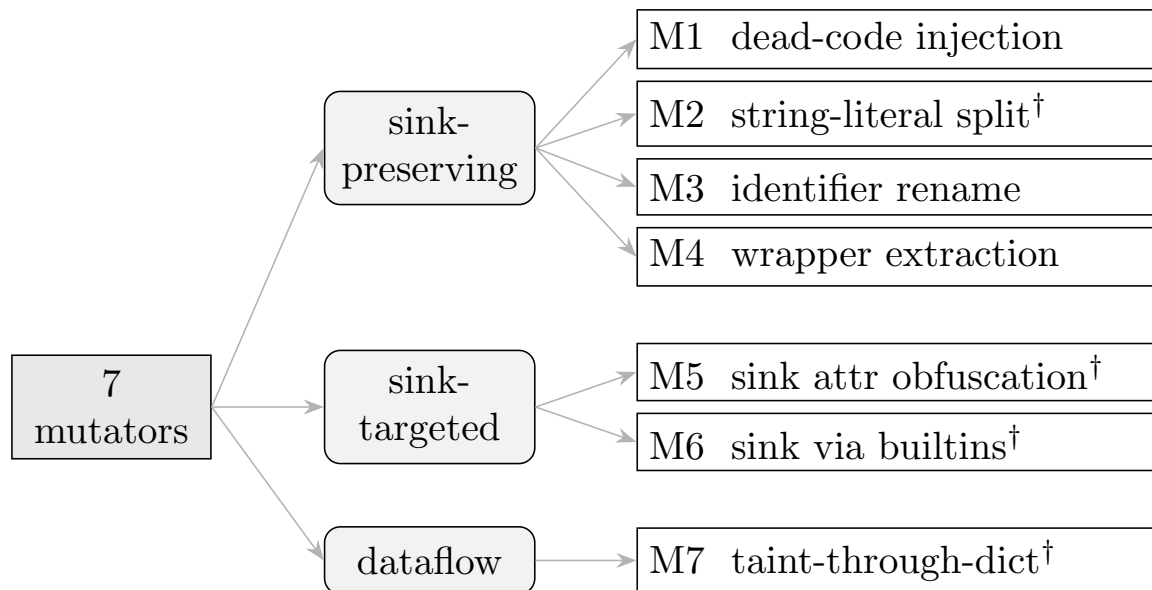


Figure 5.1: The seven mutators grouped by the detector shortcut each family probes. Mutators marked [†] are held out of training-time augmentation and used only as test-time robustness probes.

5.2.4 Metrics Module

The **metrics module** scores each detector one-versus-rest per CWE and computes five bounded measures: macro-F1 across the seven classes, detection MCC on the binary vulnerable-versus-safe collapse, Severity-Weighted Recall on the CVSS-scored positives, Hierarchical CWE Macro-Accuracy using Wu-Palmer partial credit on a hand-coded CWE subtree, and weighted Cohen’s κ . It also computes the robustness drop as clean macro-F1 minus mutated macro-F1 over the same positives.

5.2.5 Detector Runners

Each **detector runner** adapts one tool to a common interface: it takes a sample, invokes the tool, and returns a per-CWE verdict. The SAST runners invoke Bandit and Semgrep as subprocesses and parse their findings. The whole-project runners invoke Snyk Code and CodeQL. The LLM runners send a fixed prompt to each model API at temperature zero. The trained runner loads the fine-tuned GraphCodeBERT checkpoint and returns its predicted class. A uniform interface lets the harness score every tier with the same metrics module.

5.3 Summary

The detailed design organizes the system into three subsystems of cooperating modules: validation, mutation, and evaluation. The validation modules establish label quality,

the negative builder and mutator suite create the safe twins and the robustness probes, and the metrics module and detector runners produce comparable scores. Chapter 6 describes how these modules are implemented.





Chapter 6

Implementation

CHAPTER 6

IMPLEMENTATION

This chapter describes how the design is realized. It covers the programming language and platform, the code conventions, the difficulties met during development, the dataset and the implementation of each validation layer, and the algorithms behind the mutators and the trained detector. Core procedures are shown as code listings, and the full set of mutator transformations is collected in Appendix C. The chapter closes with a summary.

6.1 Programming Language Selection

The system is implemented in Python 3.11. The choice follows the domain. The benchmark analyzes Python code, so the sink patterns, the AST transformations, and the semantics-preservation checks operate in the same language they target. The standard `ast` module provides a stable syntax tree that the mutators and the negative builder rewrite node by node, and `ast.unparse` regenerates source from a modified tree, which makes a round trip from source to tree and back the basic unit of every transformation. The machine learning stack the trained detector needs, PyTorch and the HuggingFace Transformers library, is native to Python, so the whole pipeline lives in one language and one runtime. Regular expressions back the sink-presence filter, and the `difflib`-style line comparison backs the diff filter, both from the standard library.

6.2 Platform Selection

The pipeline and harness run on a single Linux or macOS workstation. The SAST detectors (Bandit 1.9.4, Semgrep 1.163.0) and the whole-project detectors (Snyk Code 1.1305.0, CodeQL) run as local subprocesses, and each runner parses the tool's native output format into a common per-CWE verdict. The LLM detectors (Claude Sonnet 4.6, GPT-4o, DeepSeek R1) run through their hosted APIs at temperature zero. The trained detector is fine-tuned on a single CUDA-capable GPU. Source retrieval uses Git for fix commits and SQLite for the CVEfixes database.

6.3 Code Conventions

6.3.1 Naming Convention

Modules are named for their role in the pipeline: the scraper, the sink-presence filter, the audit driver, the diff filter, the negative builder, the splitter, the mutators, the metrics

module, and the detector runners. Each sample carries a stable identifier that threads through every stage, so a prediction in the evaluation logs can be traced back to the source file and the validation decision that admitted it. CWE classes are referred to by their numeric code throughout, and mutators by their family and index (M1 to M7).

6.3.2 Artifact Schema

Each sample is stored as a paired source file and a schema-versioned metadata record. The record carries the CWE label, the matched sink pattern, the label provenance, the CVSS base score and severity, and a label-confidence tier. The tier separates HIGH-provable advisory positives whose sink line the diff filter confirmed changed, HIGH-curated canonical samples that are correct by construction, and MEDIUM-curator **vudenc** samples that are human-labeled but carry no paired fix to diff against. Versioning the schema lets the pipeline evolve the record format without invalidating earlier artifacts, and it makes every validation decision reproducible from the released files.

6.4 Difficulties Encountered and Strategies Used to Tackle Them

The central difficulty was label noise that the sink filter could not catch: a sample where the sink is present but is not the vulnerability, because the CVE fix touched a co-changed file. The strategy was the three-round audit, where each round measured the false-positive rate on disjoint samples and its findings motivated the next filter. Round 1 (77 samples) measured 52% and surfaced six recurring failure patterns:

- **Sink too broad.** A pattern matched a safe overload of the sink, for example a parameterized query counted as SQL injection.
- **Co-changed-file noise.** The sink was present but the fix that earned the CWE label modified a different file.
- **Documentation match.** The sink token appeared only in a comment or docstring.
- **Pattern conflation.** One regex matched two distinct sinks, one safe and one unsafe.
- **Inverted operation.** The matched line was the remediation, not the vulnerability.

- **Wrapper abstraction.** The real sink sat behind a project helper the pattern could not see.

Each pattern motivated a specific tightening, and two CWE classes with systematic issues were dropped: CWE-434 at 100% false positive, and CWE-798 with a single-source bias. Round 2 held at 51% because new samples surfaced co-changed-file noise on the untouched classes, which motivated the diff filter. A second difficulty was proving each mutator safe. The strategy was to skip any node a transformation could not prove safe, for example a rename that might collide with an imported name, and to require every mutated sample to re-parse.

6.5 Data Set

Positives are aggregated from five primary sources with differing retrieval methods and label quality, listed in Table 6.1. GHSA Advisory DB dominates because it indexes per-CWE advisories with explicit fix commits. The `vudenc` corpus is human-labeled but ships no paired fixed version, so it cannot be diff-validated and is retained as a lower-confidence tier. Twenty hand-curated canonical samples are textbook OWASP and SANS exemplars that serve as calibration points: a detector that misses them is broken regardless of its CVE-corpus performance.

Table 6.1: Positive data sources and counts after label validation. Hard negatives are constructed safe twins.

Source	Count	Retrieval
GHSA Advisory DB	384	per-advisory fix commit
<code>vudenc</code>	209	pre-labeled corpus
OSV / GHSA GraphQL	54	commit references
CVEfixes	58	local SQLite query
NVD-targeted	33	NVD API + keyword filter
Canonical (curated)	20	hand-written exemplars
Hard negatives	429	constructed safe twins

6.5.1 Layer 1: Sink-Presence Filter

For each CWE the filter defines a small set of high-recall regex sink patterns, and it admits a sample only if at least one pattern matches the source after comments and docstrings are stripped. The patterns match both safe and unsafe forms of the sink, so discrimination is left to Layer 3. Two CWEs use proximity gates: CWE-22 requires

an HTTP-request reference within twenty lines of the sink, because test fixtures with hardcoded paths are otherwise common false positives. Listing 6.1 shows the filter in simplified form.

```
def has_cwe_sink(code, cwe, file_path=None):
    if cwe in TEST_EXCLUSION_CWES and is_test_file(file_path):
        return False, None
    match_code = strip_comments_and_docstrings(code)
    for pat in SINK_PATTERNS[cwe]:
        for m in pat.finditer(match_code):
            if cwe in REQUIRES_REQUEST_PROXIMITY:
                if not has_request_near(code, m.start()):
                    continue
            if cwe in REQUIRES_SECURITY_CONTEXT:
                if not has_security_context_near(code, m.start()):
                    continue
            return True, pat.pattern
    return False, None
```

Figure 6.1: Layer-1 sink-presence filter (simplified). A sample is admitted only if a CWE sink pattern matches after comment and docstring stripping, subject to the per-CWE proximity gates.

Applied to the 1,982 pre-filter samples, Layer 1 dropped 749 (38%). The rejection rate varied by source from 28% (NVD) to 68% (CVEfixes), where commit-level mislabels were confirmed at scale.

6.5.2 Layer 2: Adversarial Audit

The audit ran three rounds on CWE-stratified samples, each sampling ten items per populous class and auditing rare classes in full. For each item the auditor received the identifier, source, repository, file path, the matched regex, and the code excerpt around the sink, and returned a binary verdict with a one-sentence justification, without seeing other auditors' verdicts. The audit is the mechanism that drove the pattern tightenings in the previous section.

6.5.3 Layer 3: Diff-Based Filter

The diff filter implements the evidence-anchored criterion: a positive is valid only if at least one line containing a sink-pattern match differs between `code_before` and `code_after`, after comment stripping and whitespace normalization. Listing 6.2 shows the procedure.

```

def sink_was_modified(code_before, code_after, cwe):
    patterns = SINK_PATTERNS.get(cwe)
    if not patterns or not code_after.strip():
        return None # cannot determine

    def sink_lines(code):
        out = set()
        for ln in strip_comments(code).splitlines():
            normalized = " ".join(ln.split())
            if not normalized:
                continue
            for p in patterns:
                if p.search(ln):
                    out.add(normalized)
                    break
        return out

    before = sink_lines(code_before)
    after = sink_lines(code_after)
    return (before - after) != set()

```

Figure 6.2: Layer-3 diff filter. A positive passes if at least one sink line in `code_before` is absent from `code_after`. Whitespace is normalized so that reformat-only edits do not count as real changes.

Of the 738 positives reaching Layer 3, 316 (43%) had unchanged sink lines and were rejected; the cut fell unevenly across classes (CWE-79 lost 68%, CWE-22 lost 71%). Positives without a paired fixed version, the **vudenc** tier, cannot be diff-checked and are retained at the MEDIUM confidence tier, so the rejected count does not subtract directly to the final positive total. After all three layers the corpus holds 440 positives. The combined pipeline lowered the audited false-positive rate from 52% to 26%.

6.5.4 Final Composition

The final benchmark is 440 positives plus 429 safe negatives, 869 samples in all, split 610/127/132 with a repo-group constraint that yields zero cross-split repository leakage. Table 6.2 gives the per-CWE composition. Three oversized hard negatives are kept on disk but excluded from active evaluation, leaving a 129-sample test set (67 positives, 62 negatives).

Table 6.2: Final per-CWE composition of the benchmark.

Label	Total	Train	Val	Test
CWE-89 (SQL Injection)	213	149	32	32
CWE-502 (Insecure Deserial.)	75	52	11	12
CWE-79 (XSS)	54	40	7	7
CWE-94 (Code Injection)	35	24	5	6
CWE-78 (OS Command Injection)	26	18	4	4
CWE-918 (SSRF)	22	17	2	3
CWE-22 (Path Traversal)	15	10	2	3
safe (hard negatives)	429	300	64	65
Total	869	610	127	132

6.5.5 Hard-Negative Construction

The 429 safe samples are not scraped. For each negative the builder takes a real vulnerable function and applies the canonical remediation for its CWE at the AST level, for example rewriting `yaml.load` to `yaml.safe_load` or a string-formatted SQL query to a parameterized query. The remediation preserves the surface structure and the sink name while neutralizing only the sink behavior. Each negative therefore carries a provable safe label and is a minimal-pair counterfactual: a detector that fires on the mere presence of a sink token is forced into a false positive on the twin.

6.6 Algorithms Used

6.6.1 Mutator Transformations

Each mutator is an AST transformer that walks the parsed tree, rewrites the nodes it can prove safe, and unparses the result. The transformations are deterministic given a sample identifier and seed, so the evaluation reproduces. Every mutator follows three principles: it is semantics-preserving and skips any node it cannot prove safe; it targets one shortcut, so a per-mutator accuracy drop is diagnostic; and it is realistic, so identifiers become synonyms and injected code reads like ordinary scaffolding. The three families attack distinct shortcuts:

- **Sink-preserving (M1 to M4).** Dead-code injection breaks positional reasoning, string splitting breaks substring matching, identifier rename breaks name memorization, and wrapper extraction breaks single-function analysis. All four leave the sink identifier and its arguments intact.

- **Sink-targeted (M5, M6).** M5 rewrites an attribute sink `os.system(x)` through a `getattr` lookup, and M6 rewrites a builtin sink `eval(x)` through a `builtins` dictionary lookup, so in both cases the sink token survives only inside a string literal (Appendix C gives the full rewrites). Both break token-level sink matching.
- **Dataflow (M7).** Taint-through-dict routes the sink’s first argument through a one-entry dict, `f({"a":x}["a"])`, breaking the static lineage from parameter to sink argument while leaving the sink and its value unchanged.

Three of the seven mutators are applied as online training augmentation, and the four marked as held-out probes (M2, M5, M6, M7) are used only at test time. Appendix C gives a before-and-after listing for each of the seven mutators.

6.6.2 Trained Detector

The 610-sample training split fine-tunes a dual-task GraphCodeBERT that predicts both the CWE class and the CVSS base score. Training uses a three-phase schedule. The first phase is a supervised-contrastive warmup that pulls together representations of samples sharing a CWE. The second phase applies an LDAM margin loss [38] on a class-balanced sampler, which widens the decision margin for the rare classes. The third phase is a deferred re-weighting stage that switches to Effective-Number class weights [39] once the representation has stabilized. The model has 126.6M parameters and is treated as a small-data existence proof rather than a state-of-the-art claim.

6.7 Summary

The implementation realizes the three subsystems in Python: the corpus pipeline with its three validation layers, shown here as the Layer-1 and Layer-3 listings, and the constructed negatives; the AST-based mutator suite; and the evaluation harness with its detector runners and trained model. The dataset tables fix the composition the next chapters test and evaluate. Chapter 7 describes how the implementation is tested.



Chapter 7

Software Testing

CHAPTER 7

SOFTWARE TESTING

Testing the benchmark system means testing two things: that the pipeline produces the corpus it claims to, and that the validation and mutation logic behaves as specified. Because the system is a research pipeline rather than an interactive application, much of its testing takes the form of the audit that measures label quality and the semantics-preservation checks that guard the mutators. This chapter describes the testing strategy, component testing, pipeline testing, the audit as a validation procedure, and the semantics-preservation regime for the mutators, with representative test cases.

7.1 Testing Strategy

The system is tested at three levels. Component testing checks each module against known inputs. Pipeline testing runs the stages end to end and checks the invariants that connect them. Validation testing measures the quality of the output corpus through the independent audit. The three levels answer different questions: component testing asks whether a module does what its specification says, pipeline testing asks whether the modules compose without breaking an invariant, and validation testing asks whether the labels the pipeline emits are correct. A defect at any level is caught before the corpus is released, because the audit re-measures label quality on fresh samples each round.

7.2 Component Testing

Each pipeline module is tested in isolation against known inputs.

7.2.1 Sink-Presence Filter

The sink-presence filter is tested for recall, for documentation suppression, and for the proximity gates. Recall is checked by confirming that a sample containing a known sink for its CWE is admitted. Documentation suppression is checked by confirming that a sink token appearing only in a comment or docstring does not cause admission, since the filter strips comments and docstrings before matching. The proximity gate for CWE-22 is checked by confirming that a filesystem sink without a nearby HTTP-request reference is rejected, which removes test fixtures with hardcoded paths. The filter is also tested for the test-file exclusion, which drops samples whose path marks them as test code for the CWEs where test fixtures are a known noise source.

7.2.2 Diff Filter

The diff filter is tested against paired vulnerable and fixed versions. A positive whose sink line differs between the two versions must be kept, and a positive whose sink line is unchanged, even when other lines in the file differ, must be rejected. Comment stripping and whitespace normalization are tested by confirming that a change limited to a comment or to whitespace does not count as a sink-line change, so a reformat-only commit does not pass a sample. The filter is also tested for the undetermined case: when no fixed version is available, the procedure returns a null verdict rather than a pass, which is what keeps the `vudenc` samples in the lower-confidence tier instead of admitting them as diff-validated.

7.2.3 Negative Builder

The negative builder is tested by confirming that a constructed safe twin re-parses, that it preserves the sink name, and that the remediation neutralizes the sink behavior. A twin produced from a `yaml.load` sink must contain `yaml.safe_load` and must still parse as valid Python. A twin produced from a string-formatted query must use parameter binding rather than string concatenation. The test confirms that the surface structure around the sink is otherwise unchanged, which is what makes the twin a minimal pair against its vulnerable original.

7.2.4 Mutator Semantics Preservation

Every mutator is tested for the property that defines it: that the mutated sample re-parses and that its runtime behavior matches the original. Re-parsing is checked for every mutated sample in the test set, so a transformation that produces syntactically invalid Python is caught before evaluation. Behavior preservation is checked per transformation against the Python evaluation rules. The tests confirm the specific safety conditions each mutator depends on:

- Dead-code injection inserts only side-effect-free statements (`if False: pass`, an unused assignment, a zero-iteration loop), so the function's return value cannot change.
- String splitting skips docstrings and type-hint strings, so `help()` output and runtime type introspection are unaffected, and it splits only the literal text between f-string substitutions.

- Identifier rename skips `self`, `cls`, builtins, imported names, names declared `global` or `nonlocal`, the function’s own name, and any token inside a string literal, so dispatch, recursion, and string data are preserved.
- Wrapper extraction forwards arguments with `*_a`, `**_kw`, generates an `async def` wrapper for an awaited sink, and inserts the helper after a docstring rather than before it.
- The sink-targeted and dataflow mutators are checked to leave the runtime value reaching the sink bitwise identical to the original argument.

A node a mutator cannot prove safe to transform is skipped, which is tested by confirming that a rename colliding with an imported name is not applied and that the surrounding code is left unchanged.

7.3 Pipeline Testing

Pipeline testing runs the stages end to end and checks the invariants. The split is tested for the repo-group constraint by confirming zero cross-split repository leakage, so no repository contributes samples to more than one split. The corpus is tested for schema conformance by confirming that every sample carries a CWE label, a matched sink pattern, a label provenance, a CVSS severity, and a label-confidence tier. The mutation stage is tested by confirming that each test sample yields the expected seven variants and that each variant re-parses. The per-CVE sample cap is tested by confirming that no single CVE contributes more than three near-identical files, which prevents one high-impact advisory from dominating a class.

7.4 Validation Testing through the Audit

The audit is the system’s primary validation test, and it follows the controlled-measurement discipline of empirical software engineering [40]. Three rounds of independent audit measured the false-positive rate of the labels on disjoint CWE-stratified samples. Round 1 (77 samples) measured 52% and surfaced six recurring failure patterns, each of which became a fix in the next round. Round 2 held at 51% after the pattern tightenings, because new samples surfaced co-changed-file noise on the classes the tightenings had not touched, which motivated the diff filter. Round 3, on the diff-filtered data, measured 26%. The audit is adversarial by construction: reviewers see only the sample

and its evidence, not each other’s verdicts, so a mislabeled sample is more likely to be caught than confirmed. The falling false-positive rate across rounds is the evidence that the validation pipeline works, and the residual 26% is reported rather than hidden.

7.5 Test Cases

Table 7.1 lists representative test cases and their outcomes.

Table 7.1: Representative test cases for the benchmark pipeline.

#	Test	Expected result	Status
1	Sink in code only	Sample admitted	Pass
2	Sink in docstring only	Sample rejected	Pass
3	CWE-22 sink, no HTTP source nearby	Sample rejected	Pass
4	Sink in a test-file path	Sample excluded	Pass
5	Sink line changed in fix	Positive kept	Pass
6	Sink line unchanged in fix	Positive rejected	Pass
7	Comment-only change in fix	Positive rejected	Pass
8	No fixed version available	Undetermined, kept as MEDIUM tier	Pass
9	Safe twin re-parses	Valid Python, sink name kept	Pass
10	Mutated sample re-parses	Valid Python for all 7 variants	Pass
11	Rename collides with import	Node skipped, code unchanged	Pass
12	Repo-group split	Zero cross-split leakage	Pass
13	Per-CVE cap	At most 3 files per CVE	Pass

7.6 Summary

The benchmark system is tested at the component level for each module, at the pipeline level for its invariants, and at the validation level through the three-round audit that measures label quality. The semantics-preservation checks guard the mutators against changing program behavior, the repo-group check guards the split against leakage, and the per-CVE cap guards a class against single-CVE domination. The test cases in Table 7.1 confirm the specified behavior across all three levels. Chapter 8 reports the experimental results that this validated pipeline makes possible.



Chapter 8

Experimental Results

CHAPTER 8

EXPERIMENTAL RESULTS

This chapter reports the evaluation of seven detectors across four tiers on the benchmark. It states the research questions and the experimental setup, then presents the headline results, the per-CWE detection on clean code, the adversarial robustness results, and a per-tier analysis of the LLMs, the trained model, and the whole-project detectors. It closes with the effect of the label filter and a summary.

8.1 Research Questions and Setup

The evaluation answers three questions. **RQ1:** how accurately do the detectors classify the seven sink-shaped CWE classes on clean code? **RQ2:** how does each detector’s accuracy change under a semantics-preserving mutator, and which mutator exposes the most brittleness? **RQ3:** does the diff-based label filter change the picture a detector evaluation paints? The purpose is to show what the benchmark measures, not to rank detectors.

The held-out test set is 129 samples (67 positives across the seven classes, 62 safe negatives). Every detector is scored one-versus-rest per CWE under five bounded measures: macro-F1, detection MCC on the binary vulnerable-versus-safe collapse, Severity-Weighted Recall (SWR) on the 27 CVSS-scored positives, Hierarchical CWE Macro-Accuracy (HCMA), and weighted Cohen’s κ . The robustness drop is clean macro-F1 minus mutated macro-F1 over the 67 positives. The seven detectors are Bandit and Semgrep (pattern-matching SAST), Snyc Code and CodeQL (whole-project dataflow), Claude Sonnet 4.6, GPT-4o, and DeepSeek R1 (LLMs), and a fine-tuned GraphCodeBERT.

8.2 Results Analysis

8.2.1 Headline Results

Table 8.1 reports the seven detectors. No single tool leads on every measure. DeepSeek R1 takes the top macro-F1 (0.596); Sonnet 4.6 leads SWR (0.903) and HCMA (0.805); the trained GraphCodeBERT leads detection MCC (+0.556); and Bandit edges weighted κ (+0.567). The two LLMs Sonnet and GPT-4o sit below the SAST and learned tiers on MCC because they over-flag hard negatives as CWE-22 and CWE-918. Snyc Code is an outlier on every detection metric (macro-F1 0.063), which is read as a property of the

single-function format rather than a ranking.

Table 8.1: Detector results on the 129-sample clean test set. Macro-F1 is one-versus-rest across seven CWEs; MCC is on the binary vulnerable/safe collapse; SWR is severity-weighted recall on the 27 CVSS-scored positives; HCMA is hierarchical accuracy; κ_w is weighted Cohen’s κ ; Comp. is macro-F1 on the composed mutator; $\Delta F1^*$ is the worst-case single-mutator drop (positives only).

Detector	macro-F1	MCC	SWR	HCMA	κ_w	Comp.	$\Delta F1^*$
Bandit	0.510	+0.478	0.745	0.564	+0.567	0.589	+0.187
Semgrep	0.315	+0.157	0.634	0.353	+0.227	0.375	+0.132
Snyk Code	0.063	−0.235	0.143	0.069	−0.097	0.088	+0.036
Claude Sonnet 4.6	0.585	+0.117	0.903	0.805	+0.396	0.867	0.000
GPT-4o	0.356	+0.047	0.420	0.419	+0.295	0.565	+0.026
DeepSeek R1	0.596	+0.410	0.691	0.646	+0.539	0.567	+0.068
GraphCodeBERT (ours)	0.524	+0.556	0.870	0.547	+0.548	0.603	0.000

The hierarchical and agreement columns sharpen the comparison between Bandit and the trained detector. On macro-F1 Bandit (0.510) trails GraphCodeBERT (0.524) by 0.014, but on HCMA the gap inverts and Bandit (0.564) edges ahead (0.547). Bandit’s errors are dominated by close-sibling mistakes inside the injection subtree, which Wu-Palmer credits as partial matches but macro-F1 scores as flat misses. Weighted κ agrees: Bandit (+0.567) and GraphCodeBERT (+0.548) are within 0.019 of each other. Bandit and the trained detector are therefore comparable on clean code under any partial-credit metric.

8.2.2 Detection on Clean Code (RQ1)

Among the SAST and learned tiers, detectors diverge most on SQL injection: Bandit reaches F1 0.80 on CWE-89 by lexically matching inline SQL, GraphCodeBERT 0.63, and Semgrep only 0.06 because its taint-shape rules require a framework-recognizable source-to-sink shape absent from the fragmentary samples. CWE-22 scores zero for all three (no usable rule, too few training samples), and CWE-918 separates the tiers: both SAST tools score zero while GraphCodeBERT recovers F1 0.57. Table 8.2 gives the per-class breakdown.

8.2.3 Adversarial Robustness (RQ2)

The mutator families split the tiers apart, shown in Table 8.3. The two pattern-matching SAST tools show a robustness cliff on the sink-targeted family: Bandit drops

Table 8.2: Per-CWE F1 on the clean test set (n = positive test samples). GCB is the trained GraphCodeBERT detector.

CWE	n	Bandit	Semgrep	GCB
CWE-22 Path Traversal	3	0.00	0.00	0.00
CWE-78 OS Command Injection	4	0.38	0.29	0.40
CWE-79 Cross-site Scripting	7	0.60	0.33	0.67
CWE-89 SQL Injection	32	0.80	0.06	0.63
CWE-94 Code Injection	6	0.92	0.80	0.57
CWE-502 Insecure Deserial.	12	0.83	0.73	0.83
CWE-918 SSRF	3	0.00	0.00	0.57
Macro-F1 (positives + safe)		0.51	0.32	0.52

0.187 macro-F1 under attribute obfuscation and 0.143 under builtins indirection; Semgrep drops 0.109 and 0.132. The cliff replicates across two mutators attacking disjoint sink classes, so it is a property of lexical sink-matching rather than a single trick. The trained GraphCodeBERT detector is invariant to every mutator ($|\Delta F1| \leq 0.036$), as are all three LLMs. All detectors are invariant to the four sink-preserving mutators and to the dataflow mutator, the latter because none of the current detectors performs dataflow analysis, so M7 remains a coverage slot for future dataflow-aware tools.

Table 8.3: Macro-F1 by variant over the 67 positives. DC/SS/VR/WE: sink-preserving; SAO/SVG: sink-targeted; TTD: dataflow; Comp: composed.

Tool	Clean	DC	SS	VR	WE	SAO	SVG	TTD	Comp
Bandit	0.59	0.59	0.59	0.59	0.59	0.40	0.45	0.59	0.58
Semgrep	0.38	0.38	0.38	0.38	0.38	0.28	0.25	0.37	0.38
GraphCodeBERT	0.57	0.59	0.60	0.60	0.61	0.60	0.57	0.60	0.60

Table 8.4 reports the drops grouped by family in both macro-F1 and severity-weighted recall. The SAST cliff is visible on the sink-targeted columns (Bandit loses up to 0.347 SWR) and absent on the learned detector.

8.2.4 Per-CWE Validation Progression

Figure 8.1 shows the per-CWE audit false-positive rate across the three validation rounds. The diff filter collapses the co-changed-file classes (CWE-502, CWE-94) to 0%, while the residual noise concentrates on CWE-89 and CWE-79, where `vudenc` samples are retained without a paired fix to diff against.

Table 8.4: Robustness drop (clean – mutator) by family. Sink-preserve is the mean of the four sink-preserving mutators; SAO, SVG, and TTD are individual. Δ SWR is the drop in severity-weighted recall.

Tool	Sink-preserve		SAO		SVG		TTD	
	Δ F1	Δ SWR	Δ F1	Δ SWR	Δ F1	Δ SWR	Δ F1	Δ SWR
Bandit	+0.001	.000	+0.187	+0.347	+0.143	+0.246	.000	.000
Semgrep	+0.005	.000	+0.109	+0.196	+0.132	+0.246	+0.018	−0.029
GraphCodeBERT	−0.036	+0.009	−0.035	+0.034	.000	.000	−0.030	.000

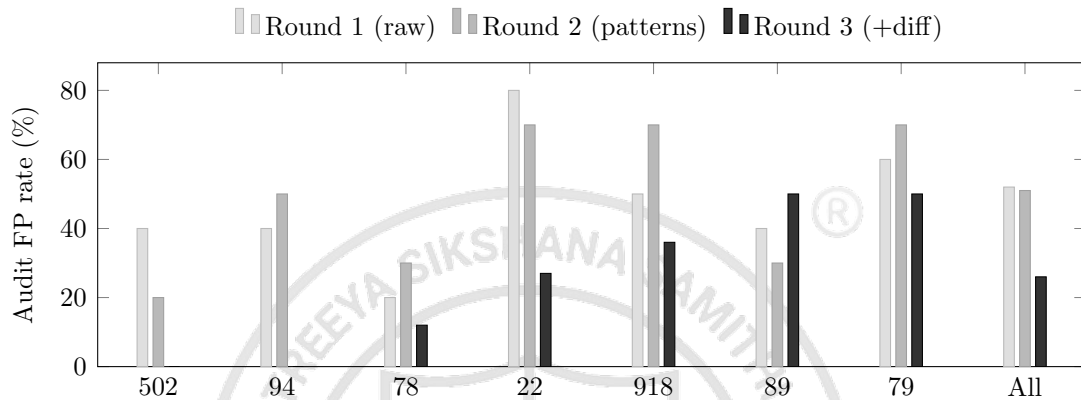


Figure 8.1: Per-CWE audit false-positive rate across the three validation rounds (x-axis labels are CWE numbers; *All* is the aggregate). The diff filter collapses the co-changed-file classes to 0% while the residual concentrates on CWE-89 and CWE-79.

8.3 Detailed Analysis

8.3.1 LLM Detectors

The three LLMs differ. DeepSeek R1 posts the highest macro-F1 in the table (0.596) and the highest CWE-89 F1 of any detector (0.935); its MCC (+0.410) is the best of the three LLMs. Sonnet 4.6 ranks just below on macro-F1 (0.585) and takes the SWR (0.903) and HCMA (0.805) leads of the whole table, but its MCC (+0.117) is low because it over-flags hard negatives as CWE-22 (15 false positives) and CWE-918 (17). GPT-4o is the weakest LLM (macro-F1 0.356, MCC +0.047 near chance): it reaches F1 0.915 on CWE-89 but scores zero on CWE-94 and over-applies CWE-22 and CWE-918 to safe code, a miscalibration rather than a uniform weakness. All three are invariant to the mutators, which confirms that the sink-token cliff is a property of lexical SAST, not of the mutator suite.

8.3.2 Trained-Model Detector

The fine-tuned GraphCodeBERT reaches macro-F1 0.524 and the table-leading detection MCC (+0.556), which reflects a lower false-positive rate on hard negatives than either SAST tool or any LLM. It is invariant to every mutator, including the sink-targeted family that breaks the SAST tools, because it draws on semantic features beyond the sink identifier. This is a defensible small-data existence proof: 610 samples is small for a 126.6M-parameter model, so the result shows that a benchmark-tuned learned detector can compete with mature baselines on Python sink-shaped CWEs, not that it is state of the art.

8.3.3 Whole-Project Dataflow Detectors

Snyk Code is built for whole-application analysis but is scored here on single-function extracts; it flags 1 of 32 CWE-89 positives and produces many false positives on the safe class, because the framework source-to-sink lineage it tracks is absent from the extracts. CodeQL returned zero findings for the same reason: its source model requires application entry points the extracts do not contain. This is a property of the benchmark format, not the tools, and both detector implementations are kept in the released harness for a future application-level benchmark.

8.3.4 Effect of the Label Filter (RQ3)

The numbers above use the diff-filtered labels (26% audited false positive). Run against unfiltered labels, every tool's apparent precision falls by construction, because a tool that correctly declines a mislabeled sample is charged a false negative. The filter therefore does not flatter the tools; it removes samples on which a correct detector and a wrong label disagree.

8.4 Summary

Across seven detectors in four tiers, no single tool leads on every measure, and the sink-token mutator family is the axis that separates pattern-matching SAST, which drops up to 0.187 macro-F1, from a learned detector invariant to every mutator. The validation progression in Figure 8.1 shows where label noise was removed, and the label-filter analysis shows that the filter tightens rather than inflates the scores. Chapter 9 draws conclusions and states limitations and future work.



Chapter 9

Conclusion

CHAPTER 9

CONCLUSION

This project presented an audit-validated benchmark for Python vulnerability detection of 869 samples across seven sink-shaped CWE classes, built with a three-layer label-validation methodology that halved the audited false-positive rate from 52% to 26% and a diff-based filter that anchors each positive on a modified sink line. Paired with seven semantics-preserving mutators, the benchmark evaluates detection and robustness jointly. Across seven detectors in four tiers, no single tool led on every measure, and the sink-token mutator family proved to be the axis that separates pattern-matching SAST, which drops up to 0.187 macro-F1, from a learned detector invariant to every mutator. The objectives stated in Chapter 1 were met: the corpus was assembled and validated, the mutator suite was built, and the seven detectors were evaluated on clean and mutated code under five bounded metrics.

9.1 Limitations

The principal limitations follow the standard validity categories.

Construct validity. The label set is not noise-free. An absolute score on this benchmark inherits the 26% audited residual and should not be read as a tool’s true accuracy on a hypothetical clean corpus; the between-tool comparison on the same labels is the robust reading. The sink-presence filter is itself a definition of a correct label, so a detector using the same sink regexes inherits a measurement advantage; the full pattern set, the audit reports, and a per-sample rejection manifest are released so this dependence is auditable.

Internal validity. Single-CVE clustering is mitigated by a per-CVE sample cap of three and by the repo-group split, which yields zero cross-split leakage. The evaluation uses one splitter seed and one run per detector; this is sound for the deterministic SAST tools and the fixed-checkpoint model, and the LLMs run at temperature zero, so per-call drift is small but not bit-for-bit zero.

External validity. The benchmark is Python-specific and limited to sink-shaped CWEs, so it does not measure structural-CWE detection and is not a comprehensive Python ranking. The scraped corpus over-represents popular projects (Django, Flask, FastAPI), which the 20 canonical samples offset only in part. The single-file extract

format is unfavorable to whole-project dataflow detectors.

Conclusion validity. The 129-sample test set is small. Per-class F1 intervals are wide for CWE-22, CWE-78, and CWE-918, where a single prediction flip moves F1 by 14 to 33 points, so an absolute macro-F1 change below 0.04 lies near the noise floor. The learned-detector invariance is a property of one GraphCodeBERT checkpoint, not of learned detectors in general.

9.2 Future Enhancement

Several extensions follow from the limitations:

- An application-level benchmark format that supplies entry points to the whole-project dataflow detectors, so CodeQL and Snyk Code can be measured on the analysis they are built for.
- A multi-seed, multi-model protocol with paired bootstrap intervals to place confidence bounds on the robustness conclusions.
- A dynamic-dispatch mutator axis (dispatch through a runtime-variable attribute name) to test whether the learned detector’s invariance holds on an axis not yet exercised.
- Re-deriving the sink pattern set for other languages, so the validation methodology can extend beyond Python.
- Expanding the corpus to under-represented and unmaintained projects to reduce the popularity bias.

The benchmark, its audit reports, the rejection manifests, and the tool runners are released to support replication and extension to additional languages, CWE classes, and dataflow-aware detectors. By measuring label quality and robustness alongside detection accuracy, the work gives a more honest basis for comparing Python vulnerability detectors and a foundation for further research.



Appendix A

Plagiarism report

APPENDIX A

PLAGIARISM REPORT

Attach the screenshot of plagiarism report front page.





Appendix B

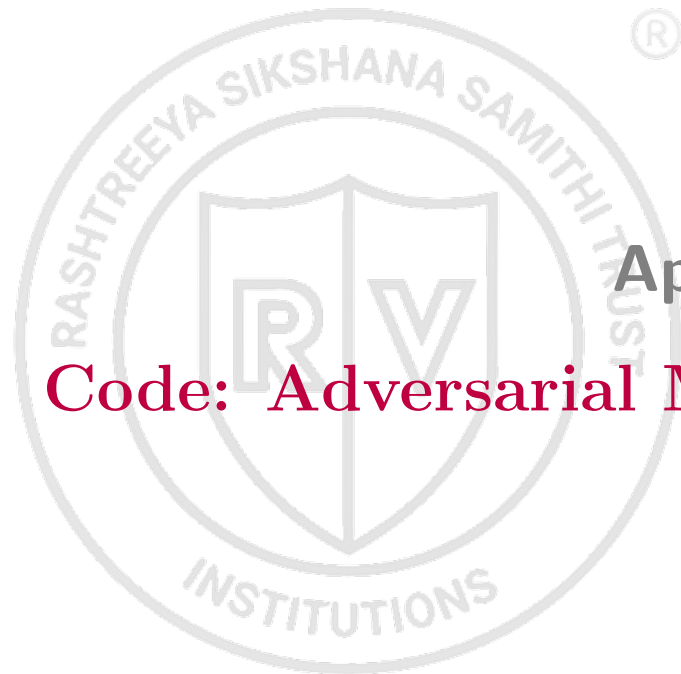
Publication details

APPENDIX B

PUBLICATION DETAILS

Attach the screenshot of either the Acknowledgment of the publication / Proof of Acceptance / Proof of Registration.





Appendix C

Code: Adversarial Mutators

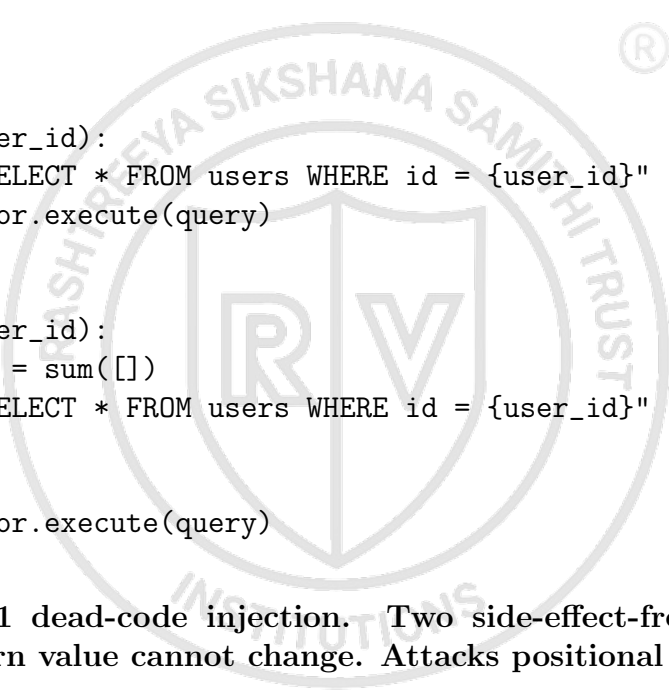
APPENDIX C

CODE: ADVERSARIAL MUTATORS

This appendix gives a before-and-after listing for each of the seven semantics-preserving mutators described in Chapter 6. Every transformation leaves runtime behavior unchanged while breaking one detector shortcut. The two core validation procedures, the Layer-1 sink-presence filter and the Layer-3 diff filter, appear as Listings 6.1 and 6.2 in Chapter 6.

C.1 Sink-Preserving Family (M1 to M4)

These mutators alter the code around the sink but leave the sink identifier and its arguments intact.



```
# Before
def get_user(user_id):
    query = f"SELECT * FROM users WHERE id = {user_id}"
    return cursor.execute(query)

# After
def get_user(user_id):
    _unused_var = sum([])
    query = f"SELECT * FROM users WHERE id = {user_id}"
    if False:
        pass
    return cursor.execute(query)
```

Figure C.1: M1 dead-code injection. Two side-effect-free statements are added; the return value cannot change. Attacks positional reasoning.

```
# Before
return cursor.execute(f"SELECT * FROM users WHERE id = {uid}")

# After
return cursor.execute(
    "SEL" + "ECT * FROM" + " users WHERE id = " + f"{uid}"
)
```

Figure C.2: M2 string-literal splitting. The substring "SELECT" no longer appears in the source, but the runtime SQL string is byte-identical. Attacks substring matching. Held out as a test-time probe.

```

# Before
def get_user(user_id):
    query = f"SELECT * FROM users WHERE id = {user_id}"
    return cursor.execute(query)

# After
def get_user(account_pk):
    sql = f"SELECT * FROM users WHERE id = {account_pk}"
    return cursor.execute(sql)

```

Figure C.3: M3 identifier rename. Locals and parameters are renamed to synonyms in all their references. Attacks identifier memorization.

```

# Before
def get_user(user_id):
    query = f"SELECT * FROM users WHERE id = {user_id}"
    return cursor.execute(query)

# After
def get_user(user_id):
    def _wrapped_execute_138(*_a, **_kw):
        return cursor.execute(*_a, **_kw)

    query = f"SELECT * FROM users WHERE id = {user_id}"
    return _wrapped_execute_138(query)

```

Figure C.4: M4 wrapper extraction. The sink call is moved into a one-line nested helper that forwards arguments. Attacks single-function analysis.

C.2 Sink-Targeted Family (M5, M6)

These mutators rewrite the sink call itself, so the sink token survives only inside a string literal. Both are held out as test-time probes.

```

# Before
def run_cmd(user_arg):
    return os.system("ls " + user_arg)

# After
def run_cmd(user_arg):
    return getattr(os, "system")("ls " + user_arg)

```

Figure C.5: M5 sink attribute obfuscation. An attribute sink value.attr(args) becomes getattr(value, "attr")(args). Attacks token-level sink matching for CWE-78, 89, 918, and 502.

```
# Before
def render_template(spec):
    return eval(spec)

# After
def render_template(spec):
    return __import__("builtins").__dict__["eval"](spec)
```

Figure C.6: M6 sink via globals. A builtin sink `eval(arg)` is rewritten through a `builtins` dictionary lookup, as shown above, so the sink token survives only as a string literal. Attacks the same token-level match on the CWE-94 bare-builtin position.

C.3 Dataflow Family (M7)

This mutator leaves the sink and its argument value unchanged and instead breaks the static lineage from source to sink.

```
# Before
def get_user(user_id):
    return cursor.execute(
        "SELECT * FROM users WHERE id = " + user_id
    )

# After
def get_user(user_id):
    return cursor.execute(
        {"_a": "SELECT * FROM users WHERE id = " + user_id}["_a"]
    )
```

Figure C.7: M7 taint-through-dict. The sink's first argument is routed through a one-entry dict that is constructed and immediately re-extracted, so the value is bitwise identical. Attacks taint tracking that does not propagate through dict reads. Held out as a test-time probe.

BIBLIOGRAPHY

- [1] R. Croft, M. A. Babar, and M. M. Kholoosi, “Data quality for software vulnerability datasets,” in *Proceedings of the 45th International Conference on Software Engineering (ICSE ’23)*, IEEE, 2023, pp. 121–133. DOI: 10.1109/ICSE48619.2023.00022
- [2] Y. Li, Y. Zhang, and et al., *CleanVul: Automatic function-level vulnerability detection in code commits using LLM heuristics*, arXiv:2411.17274, 2024.
- [3] Y. Ding et al., “Vulnerability detection with code language models: How far are we?” In *Proceedings of the 47th International Conference on Software Engineering (ICSE ’25)*, arXiv:2403.18624, 2025.
- [4] S. Ullah, M. Han, S. Pujar, H. Pearce, A. Coskun, and G. Stringhini, “LLMs cannot reliably identify and reason about security vulnerabilities (yet?): A comprehensive evaluation, framework, and benchmarks,” in *2024 IEEE Symposium on Security and Privacy (S&P)*, arXiv:2312.12575, 2024.
- [5] MITRE Corporation, *CWE Top 25 Most Dangerous Software Weaknesses*, <https://cwe.mitre.org/top25/>, 2024.
- [6] J. Fan, Y. Li, S. Wang, and T. N. Nguyen, “A C/C++ code vulnerability dataset with code changes and CVE summaries,” in *Proceedings of the 17th International Conference on Mining Software Repositories (MSR ’20)*, ACM, 2020, pp. 508–512. DOI: 10.1145/3379597.3387501
- [7] Y. Chen, Y. Ding, L. Alowain, X. Chen, and D. Wagner, “DiverseVul: A new vulnerable source code dataset for deep learning based vulnerability detection,” in *Proceedings of the 26th International Symposium on Research in Attacks, Intrusions and Defenses (RAID ’23)*, ACM, 2023, pp. 654–668. DOI: 10.1145/3607199.3607242
- [8] S. Chakraborty, R. Krishna, Y. Ding, and B. Ray, “Deep learning based vulnerability detection: Are we there yet?” *IEEE Transactions on Software Engineering*, vol. 48, no. 9, pp. 3280–3296, 2022. DOI: 10.1109/TSE.2021.3087402

- [9] R. A. Dubniczky, K. Z. Horvát, T. Bisztray, M. A. Ferrag, L. C. Cordeiro, and N. Tihanyi, “CASTLE: Benchmarking dataset for static code analyzers and LLMs towards CWE detection,” in *Proceedings of the International Symposium on Theoretical Aspects of Software Engineering (TASE '25)*, 2025.
- [10] PyCQA, *Bandit: A security linter for Python*, <https://bandit.readthedocs.io/>, 2024.
- [11] Semgrep, Inc., *Semgrep: Lightweight static analysis for many languages*, <https://semgrep.dev/>, 2024.
- [12] Pylint contributors, *Pylint: A static code analyzer for Python*, <https://pylint.readthedocs.io/>, 2024.
- [13] GitHub, Inc., *CodeQL: Semantic code analysis*, <https://codeql.github.com/>, 2024.
- [14] A. Khare, S. Dutta, Z. Li, A. Solko-Breslin, R. Alur, and M. Naik, *Understanding the effectiveness of large language models in detecting security vulnerabilities*, arXiv:2311.16169, 2023.
- [15] B. Steenhoek, M. M. Rahman, M. K. Roy, M. S. Alam, E. T. Barr, and W. Le, *A comprehensive study of the capabilities of large language models for vulnerability detection*, arXiv:2403.17218, 2024.
- [16] H. Pearce, B. Ahmad, B. Tan, B. Dolan-Gavitt, and R. Karri, “Asleep at the keyboard? assessing the security of GitHub Copilot’s code contributions,” in *2022 IEEE Symposium on Security and Privacy (S&P)*, 2022, pp. 754–768. DOI: 10.1109/SP46214.2022.9833571
- [17] D. Guo et al., “GraphCodeBERT: Pre-training code representations with data flow,” in *International Conference on Learning Representations (ICLR)*, 2021.
- [18] Z. Feng et al., “CodeBERT: A pre-trained model for programming and natural languages,” in *Findings of the Association for Computational Linguistics: EMNLP 2020*, 2020, pp. 1536–1547.
- [19] Z. Li et al., “VulDeePecker: A deep learning-based system for vulnerability detection,” in *Proceedings of the 25th Annual Network and Distributed System Security Symposium (NDSS)*, 2018. DOI: 10.14722/ndss.2018.23158

- [20] R. L. Russell et al., “Automated vulnerability detection in source code using deep representation learning,” in *17th IEEE International Conference on Machine Learning and Applications (ICMLA)*, 2018, pp. 757–762. DOI: 10.1109/ICMLA.2018.00120
- [21] N. Yefet, U. Alon, and E. Yahav, “Adversarial examples for models of code,” *Proceedings of the ACM on Programming Languages (OOPSLA)*, vol. 4, pp. 1–30, 2020. DOI: 10.1145/3428230
- [22] P. Bielik and M. Vechev, “Adversarial robustness for code,” in *Proceedings of the 37th International Conference on Machine Learning (ICML)*, 2020, pp. 896–907.
- [23] J. Henkel, G. Ramakrishnan, Z. Wang, A. Albarghouthi, S. Jha, and T. Reps, “Semantic robustness of models of source code,” in *2022 IEEE International Conference on Software Analysis, Evolution and Reengineering (SANER)*, IEEE, 2022, pp. 526–537.
- [24] M. Sokolova and G. Lapalme, “A systematic analysis of performance measures for classification tasks,” *Information Processing & Management*, vol. 45, no. 4, pp. 427–437, 2009. DOI: 10.1016/j.ipm.2009.03.002
- [25] D. Chicco and G. Jurman, “The advantages of the Matthews correlation coefficient (MCC) over F1 score and accuracy in binary classification evaluation,” *BMC Genomics*, vol. 21, no. 6, 2020. DOI: 10.1186/s12864-019-6413-7
- [26] Z. Wu and M. Palmer, “Verb semantics and lexical selection,” in *Proceedings of the 32nd Annual Meeting of the Association for Computational Linguistics (ACL)*, 1994, pp. 133–138.
- [27] J. Cohen, “Weighted kappa: Nominal scale agreement with provision for scaled disagreement or partial credit,” *Psychological Bulletin*, vol. 70, no. 4, pp. 213–220, 1968.
- [28] G. Bhandari, A. Naseer, and L. Moonen, “CVEfixes: Automated collection of vulnerabilities and their fixes from open-source software,” in *Proceedings of the 17th International Conference on Predictive Models and Data Analytics in Software Engineering (PROMISE ’21)*, ACM, 2021, pp. 30–39. DOI: 10.1145/3475960.3475985

- [29] G. Nikitopoulos, K. Dritsa, P. Louridas, and D. Mitropoulos, “CrossVul: A cross-language vulnerability dataset with commit data,” in *Proceedings of the 29th ACM Joint European Software Engineering Conference and Symposium on the Foundations of Software Engineering (ESEC/FSE ’21)*, ACM, 2021, pp. 1565–1569. DOI: 10.1145/3468264.3473122
- [30] National Institute of Standards and Technology, *Juliet test suite for Python, C/C++, and Java*, <https://samate.nist.gov/SARD/test-suites>, 2017.
- [31] Y. Zhou, S. Liu, J. Siow, X. Du, and Y. Liu, “Devign: Effective vulnerability identification by learning comprehensive program semantics via graph neural networks,” in *Advances in Neural Information Processing Systems (NeurIPS)*, 2019, pp. 10 197–10 207.
- [32] L. Wartschinski, Y. Noller, T. Vogel, T. Kehrer, and L. Grunske, “VUDENC: Vulnerability detection with deep learning on a natural codebase for Python,” *Information and Software Technology*, vol. 144, p. 106 809, 2022. DOI: 10.1016/j.infsof.2021.106809
- [33] Y. Zheng et al., “D2A: A dataset built for AI-based vulnerability detection methods using differential analysis,” in *2021 IEEE/ACM 43rd International Conference on Software Engineering: Software Engineering in Practice (ICSE-SEIP)*, IEEE, 2021, pp. 111–120. DOI: 10.1109/ICSE-SEIP52600.2021.00020
- [34] RealVuln authors, *RealVuln: Benchmarking vulnerability scanners on real-world Python repositories*, arXiv:2604.13764, 2026.
- [35] Z. Li, S. Dutta, and M. Naik, “IRIS: LLM-assisted static analysis for detecting security vulnerabilities,” in *International Conference on Learning Representations (ICLR)*, arXiv:2405.17238, 2025.
- [36] M. Fu and C. Tantithamthavorn, “LineVul: A transformer-based line-level vulnerability prediction,” in *Proceedings of the 19th International Conference on Mining Software Repositories (MSR ’22)*, IEEE, 2022, pp. 608–620.
- [37] J. Akhoundali, S. Rahim Nouri, K. Rietveld, and O. Gadyatskaya, “MoreFixes: A large-scale dataset of CVE fix commits mined through enhanced repository discovery,” in *Proceedings of the 20th International Conference on Predictive Models and Data Analytics in Software Engineering (PROMISE ’24)*, ACM, 2024.

- [38] K. Cao, C. Wei, A. Gaidon, N. Arechiga, and T. Ma, “Learning imbalanced datasets with label-distribution-aware margin loss,” in *Advances in Neural Information Processing Systems (NeurIPS)*, 2019, pp. 1567–1578.
- [39] Y. Cui, M. Jia, T.-Y. Lin, Y. Song, and S. Belongie, “Class-balanced loss based on effective number of samples,” in *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2019, pp. 9268–9277.
- [40] C. Wohlin, P. Runeson, M. Höst, M. C. Ohlsson, B. Regnell, and A. Wesslén, *Experimentation in Software Engineering*. Springer, 2012. DOI: 10.1007/978-3-642-29044-2

